

File: settings.h

Directory: C:/Users/sense/Downloads/grbl-Mega-5X-edge/grbl-Mega-5X-edge/grbl

/*

settings.h - eeprom configuration handling
Part of Grbl

Copyright (c) 2017-2022 Gauthier Briere
Copyright (c) 2011-2016 Sungeun K. Jeon for Gnea Research LLC
Copyright (c) 2009-2011 Simen Svale Skogsrud

Grbl is free software: you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or
(at your option) any later version.

Grbl is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

*/

```
#ifndef settings_h
#define settings_h
```

```
#include "grbl.h"
```

```
#ifndef EEPROM_LINE_SIZE
#define EEPROM_LINE_SIZE 80
#endif
```

```
// Version of the EEPROM data. Will be used to migrate existing data from older versions of Grbl
// when firmware is upgraded. Always stored in byte 0 of eeprom
#define SETTINGS_VERSION 10 // NOTE: Check settings_reset() when moving to next version.
```

```
// Define bit flag masks for the boolean settings in settings.flag.
```

```
#define BIT_REPORT_INCHES    0
#define BIT_LASER_MODE       1
#define BIT_INVERT_ST_ENABLE 2
#define BIT_HARD_LIMIT_ENABLE 3
#define BIT_HOMING_ENABLE    4
#define BIT_SOFT_LIMIT_ENABLE 5
#define BIT_INVERT_LIMIT_PINS 6
#define BIT_INVERT_PROBE_PIN  7
```

```
#define BITFLAG_REPORT_INCHES    bit(BIT_REPORT_INCHES)
#define BITFLAG_LASER_MODE       bit(BIT_LASER_MODE)
#define BITFLAG_INVERT_ST_ENABLE bit(BIT_INVERT_ST_ENABLE)
```

```

#define BITFLAG_HARD_LIMIT_ENABLE bit(BIT_HARD_LIMIT_ENABLE)
#define BITFLAG_HOMING_ENABLE bit(BIT_HOMING_ENABLE)
#define BITFLAG_SOFT_LIMIT_ENABLE bit(BIT_SOFT_LIMIT_ENABLE)
#define BITFLAG_INVERT_LIMIT_PINS bit(BIT_INVERT_LIMIT_PINS)
#define BITFLAG_INVERT_PROBE_PIN bit(BIT_INVERT_PROBE_PIN)

// Define status reporting boolean enable bit flags in settings.status_report_mask
#define BITFLAG_RT_STATUS_POSITION_TYPE bit(0)
#define BITFLAG_RT_STATUS_BUFFER_STATE bit(1)

// Define settings restore bitflags.
#define SETTINGS_RESTORE_DEFAULTS bit(0)
#define SETTINGS_RESTORE_PARAMETERS bit(1)
#define SETTINGS_RESTORE_STARTUP_LINES bit(2)
#define SETTINGS_RESTORE_BUILD_INFO bit(3)
#ifndef SETTINGS_RESTORE_ALL
#define SETTINGS_RESTORE_ALL 0xFF // All bitflags
#endif

// Define EEPROM memory address location values for Grbl settings and parameters
#define EEPROM_ADDR_GLOBAL 1U
#define EEPROM_ADDR_PARAMETERS 512U
#define EEPROM_ADDR_STARTUP_BLOCK 768U
#define EEPROM_ADDR_BUILD_INFO 942U

// Define EEPROM address indexing for coordinate parameters
#define N_COORDINATE_SYSTEM 6 // Number of supported work coordinate systems (from index 1)
#define SETTING_INDEX_NCOORD N_COORDINATE_SYSTEM+1 // Total number of system stored (from index 0)
// NOTE: Work coordinate indices are (0=G54, 1=G55, ... , 6=G59)
#define SETTING_INDEX_G28 N_COORDINATE_SYSTEM // Home position 1
#define SETTING_INDEX_G30 N_COORDINATE_SYSTEM+1 // Home position 2
// #define SETTING_INDEX_G92 N_COORDINATE_SYSTEM+2 // Coordinate offset (G92.2,G92.3 not supported)

// Define Grbl axis settings numbering scheme. Starts at START_VAL, every INCREMENT, over N_SETTINGS.
#define AXIS_N_SETTINGS 4
#define AXIS_SETTINGS_START_VAL 100 // NOTE: Reserving settings values >= 100 for axis settings. Up to 255.
#define AXIS_SETTINGS_INCREMENT 10 // Must be greater than the number of axis settings

// Global persistent settings (Stored from byte EEPROM_ADDR_GLOBAL onwards)
typedef struct {
// Axis settings
float steps_per_mm[N_AXIS]; // Steps per units => steps_per_degree for rotationals axis (AXIS_4 and AXIS_5)
float max_rate[N_AXIS];
float acceleration[N_AXIS];
float max_travel[N_AXIS];

// Remaining Grbl settings
uint8_t pulse_microseconds;
uint8_t step_invert_mask;
uint8_t dir_invert_mask;
uint8_t stepper_idle_lock_time; // If max value 255, steppers do not disable.

```

```

uint8_t status_report_mask; // Mask to indicate desired report data.
float junction_deviation;
float arc_tolerance;

float rpm_max;
float rpm_min;
#ifdef SEPARATE_SPINDLE_LASER_PIN
    float laser_max;
    float laser_min;
#endif
#ifdef USE_OUTPUT_PWM
    float volts_max;
    float volts_min;
#endif

uint8_t flags; // Contains default boolean settings

uint8_t homing_dir_mask;
float homing_feed_rate;
float homing_seek_rate;
uint16_t homing_debounce_delay;
float homing_pulloff;
} settings_t;
extern settings_t settings;

// Initialize the configuration subsystem (load settings from EEPROM)
void settings_init();

// Helper function to clear and restore EEPROM defaults
void settings_restore(uint8_t restore_flag);

// A helper method to set new settings from command line
uint8_t settings_store_global_setting(uint8_t parameter, float value);

// Stores the protocol line variable as a startup line in EEPROM
void settings_store_startup_line(uint8_t n, char *line);

// Reads an EEPROM startup line to the protocol line variable
uint8_t settings_read_startup_line(uint8_t n, char *line);

// Stores build info user-defined string
void settings_store_build_info(char *line);

// Reads build info user-defined string
uint8_t settings_read_build_info(char *line);

// Writes selected coordinate data to EEPROM
void settings_write_coord_data(uint8_t coord_select, float *coord_data);

// Reads selected coordinate data from EEPROM
uint8_t settings_read_coord_data(uint8_t coord_select, float *coord_data);

```

```
// Returns the step pin mask according to Grbl's internal axis numbering
uint8_t get_step_pin_mask(uint8_t i);

// Returns the direction pin mask according to Grbl's internal axis numbering
uint8_t get_direction_pin_mask(uint8_t i);

// Returns the limit pin mask according to Grbl's internal axis numbering
uint8_t get_limit_pin_mask(uint8_t i);

#endif
```

File: config.h

Directory: C:/Users/sense/Downloads/grbl-Mega-5X-edge/grbl-Mega-5X-edge/grbl

```
/*
  config.h - compile time configuration
  Part of Grbl

  Copyright (c) 2017-2022 Gauthier Briere
  Copyright (c) 2012-2016 Sungeun K. Jeon for Gnea Research LLC
  Copyright (c) 2009-2011 Simen Svale Skogsrud

  Grbl is free software: you can redistribute it and/or modify
  it under the terms of the GNU General Public License as published by
  the Free Software Foundation, either version 3 of the License, or
  (at your option) any later version.

  Grbl is distributed in the hope that it will be useful,
  but WITHOUT ANY WARRANTY; without even the implied warranty of
  MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
  GNU General Public License for more details.

  You should have received a copy of the GNU General Public License
  along with Grbl. If not, see <http://www.gnu.org/licenses/>.
*/

// This file contains compile-time configurations for Grbl's internal system.
// This file is used to define the number of axis and their names,
// define homing sequences or specific needs, i.e.
// performance tuning or adjusting to non-typical machines.

// IMPORTANT: Any changes here requires a full re-compiling of the source code to propagate them.

#ifndef config_h
#define config_h

#include "grbl.h" // For Arduino IDE compatibility.
```

```

// Serial baud rate
#define BAUD_RATE 115200

// Define CPU pin map and default settings.
// NOTE: OEMs can avoid the need to maintain/update the defaults.h and cpu_map.h files and use only
// one configuration file by placing their specific defaults and pin map at the bottom of this file.
// If doing so, simply comment out these two defines and see instructions below.
#define DEFAULTS_GENERIC
#define CPU_MAP_2560_RAMPS_BOARD

//-----
// Axis definitions :
//-----
// IMPORTANT: When changing the axis definitions (axis numbers N_AXIS,
// linears axis number N_AXIS_LINEAR or axes names AXIS_*_NAME,
// don't forget to issue the reset factory defaults Grbl command: $RST=*
// if you forget the $RST=* command after change, Grbl may have
// unpredictable behavior!
//-----

#define N_AXIS 5      // Number of axes (3 to 6)
#define N_AXIS_LINEAR 4 // Number of linears axis, must be <= N_AXIS

// Axis indexing and names
#define AXIS_1 0      // Axis indexing value. Must start with 0 and be continuous.
#define AXIS_1_NAME 'X' // Axis names must be in X, Y, Z, A, B, C, U, V, W, D, E & H.
#define AXIS_2 1
#define AXIS_2_NAME 'Y'
#define AXIS_3 2
#define AXIS_3_NAME 'Z'
#if N_AXIS < 3
    #error "N_AXIS must be >= 3. N_AXIS < 3 is not implemented."
#endif
#if N_AXIS > 3
    #define AXIS_4 3
    #define AXIS_4_NAME 'Y' // Letter of axis number 4
#endif
#if N_AXIS > 4
    #define AXIS_5 4
    #define AXIS_5_NAME 'B' // Letter of axis number 5
#endif
#if N_AXIS > 5
    #define AXIS_6 5
    #define AXIS_6_NAME 'C' // Letter of axis number 6
#endif
#if N_AXIS > 6
    #error "N_AXIS must be <= 6. N_AXIS > 6 is not implemented."
#endif

// Renaming axis doesn't change their number. By default, the status report give axis values in

```

```

// the order of their number. Some graphical interface are not able to affect axis values reported
// by Grbl to the correct axis name.
// Uncomment to enable sorting of axis values by axis_names rather than by axis number. Default disabled.
// If this option is enabled, the sorting order will be X, Y, Z, U, V, W, A, B, C, D, E & H
// as defined below.
#define SORT_REPORT_BY_AXIS_NAME
#define AXIS_NAME_SORT_ORDER {'X', 'Y', 'Z', 'U', 'V', 'W', 'A', 'B', 'C', 'D', 'E', 'H'}

#ifdef SORT_REPORT_BY_AXIS_NAME
  #ifndef AXIS_NAME_SORT_ORDER
    #error You must define AXIS_NAME_SORT_ORDER to use SORT_REPORT_BY_AXIS_NAME
  #endif
#endif

// By default, Grbl report all values of each axis. When cloning axis with more than one axis with
// the same name, Grbl reports the values several times for the same axis_name if it is cloned.
// Uncomment to enable report of axis values only one time by axis_names in case of clones axis.
#define REPORT_VALUE_FOR_AXIS_NAME_ONCE

#ifdef REPORT_VALUE_FOR_AXIS_NAME_ONCE
  #ifndef SORT_REPORT_BY_AXIS_NAME
    #error You must define SORT_REPORT_BY_AXIS_NAME to use REPORT_VALUE_FOR_AXIS_NAME_ONCE
  #endif
#endif

//-----
// End of axis definitions :
//-----

//-----
// Spindle, laser and other PWM output
//-----
// Chose the spindle pin output :
// SPINDLE_PWM_ON_D8 => 0-12v 16 bits PWM on RAMPS D8 (default)
// SPINDLE_PWM_ON_D9 => 0-12v 8 bits PWM on RAMPS D9
// SPINDLE_PWM_ON_D6 => 0-5v 8bits PWM on RAMPS Servo 2 signal (Mega 2560 D6)
// Uncomment the line which correspond to your hardware
#define SPINDLE_PWM_ON_D8
//define SPINDLE_PWM_ON_D6
//define SPINDLE_PWM_ON_D9

// Spindle PWM signal inversion:
// In case of particular electronics, it may be necessary to invert the values
// of the PWM signal of the spindle. For example, if the minimum spindle
// rpm is 1 and maximum is 1000, M3S250 will output 75% instead of 25% and
// M3S750 will output 25% instead of 75%. Disabled by default
#define INVERT_SPINDLE_PWM_VALUES

// Use different spindle output pin in laser mode:
// Spindle or laser tools do not have the same hardware specifications.

```

```

// When using both spindle and laser on the same machine it will be useful
// to have spindle and laser on different pins which can deliver the
// different outputs needed.
//-----
// ! IMPORTANT: When changing the SEPARATE_SPINDLE_LASER_PIN compil option,
// don't forget to issue the reset factory defaults Grbl command: $RST=*
// if you forget the $RST=* command after change, Grbl may have
// unpredictable behavior!
//-----
// Uncomment the next line to enable this functionality (default disabled):
// #define SEPARATE_SPINDLE_LASER_PIN

#ifdef SEPARATE_SPINDLE_LASER_PIN
  // Laser PWM can be on D6 (default) or on D8 or D9.
  #define LASER_PWM_ON_D6
  // #define LASER_PWM_ON_D8
  // #define LASER_PWM_ON_D9
#else
  // Use output PWM driven by GCode command M67(Analog Output, Synchronized)
  // or GCode command M68(Analog Output, Immediate).
  //-----
  // ! IMPORTANT: When changing the USE_OUTPUT_PWM compil option,
  // don't forget to issue the reset factory defaults Grbl command: $RST=*
  // if you forget the $RST=* command after change, Grbl may have
  // unpredictable behavior!
  //-----
  // Uncomment the next line to enable the use of M67/M68 PWM output (Disabled by default).
  // #define USE_OUTPUT_PWM

  #ifdef USE_OUTPUT_PWM
    // Optional PWM can be on D9 (default) or on D8 or D6.
    // Warning ! Optional can't use the same timer than the spindle or the laser pin
    // For more information about this, see the comment of the relevant section in cpu_map.h
    #define OUTPUT_PWM_ON_D9
    // #define OUTPUT_PWM_ON_D8
    // #define OUTPUT_PWM_ON_D6
  #endif

  //-----
  // End of spindle and other PWM output
  //-----

// Define realtime command special characters. These characters are 'picked-off' directly from the
// serial read data stream and are not passed to the grbl line execution parser. Select characters
// that do not and must not exist in the streamed g-code program. ASCII control characters may be
// used, if they are available per user setup. Also, extended ASCII codes (>127), which are never in
// g-code programs, maybe selected for interface programs.
// NOTE: If changed, manually update help message in report.c.

```

```

#define CMD_RESET 0x18 // ctrl-x.
#define CMD_STATUS_REPORT '?'
#define CMD_CYCLE_START '~'
#define CMD_FEED_HOLD '!'

// NOTE: All override realtime commands must be in the extended ASCII character set, starting
// at character value 128 (0x80) and up to 255 (0xFF). If the normal set of realtime commands,
// such as status reports, feed hold, reset, and cycle start, are moved to the extended set
// space, serial.c's RX ISR will need to be modified to accomodate the change.
// #define CMD_RESET 0x80
// #define CMD_STATUS_REPORT 0x81
// #define CMD_CYCLE_START 0x82
// #define CMD_FEED_HOLD 0x83
#define CMD_SAFETY_DOOR 0x84
#define CMD_JOG_CANCEL 0x85
#define CMD_DEBUG_REPORT 0x86 // Only when DEBUG enabled, sends debug report in '{}' braces.
#define CMD_FEED_OVR_RESET 0x90 // Restores feed override value to 100%.
#define CMD_FEED_OVR_COARSE_PLUS 0x91
#define CMD_FEED_OVR_COARSE_MINUS 0x92
#define CMD_FEED_OVR_FINE_PLUS 0x93
#define CMD_FEED_OVR_FINE_MINUS 0x94
#define CMD_RAPID_OVR_RESET 0x95 // Restores rapid override value to 100%.
#define CMD_RAPID_OVR_MEDIUM 0x96
#define CMD_RAPID_OVR_LOW 0x97
// #define CMD_RAPID_OVR_EXTRA_LOW 0x98 // *NOT SUPPORTED*
#define CMD_SPINDLE_OVR_RESET 0x99 // Restores spindle override value to 100%.
#define CMD_SPINDLE_OVR_COARSE_PLUS 0x9A
#define CMD_SPINDLE_OVR_COARSE_MINUS 0x9B
#define CMD_SPINDLE_OVR_FINE_PLUS 0x9C
#define CMD_SPINDLE_OVR_FINE_MINUS 0x9D
#define CMD_SPINDLE_OVR_STOP 0x9E
#define CMD_COOLANT_FLOOD_OVR_TOGGLE 0xA0
#define CMD_COOLANT_MIST_OVR_TOGGLE 0xA1

// If homing is enabled, homing init lock sets Grbl into an alarm state upon power up. This forces
// the user to perform the homing cycle (or override the locks) before doing anything else. This is
// mainly a safety feature to remind the user to home, since position is unknown to Grbl.
#define HOMING_INIT_LOCK // Comment to disable

// Define the homing cycle patterns with bitmasks. The homing cycle first performs a search mode
// to quickly engage the limit switches, followed by a slower locate mode, and finished by a short
// pull-off motion to disengage the limit switches. The following HOMING_CYCLE_x defines are executed
// in order starting with suffix 0 and completes the homing routine for the specified-axes only. If
// an axis is omitted from the defines, it will not home, nor will the system update its position.
// Meaning that this allows for users with non-standard cartesian machines, such as a lathe (x then z,
// with no y), to configure the homing cycle behavior to their needs.
// NOTE: The homing cycle is designed to allow sharing of limit pins, if the axes are not in the same
// cycle, but this requires some pin settings changes in cpu_map.h file. For example, the default homing
// cycle can share the Z limit pin with either X or Y limit pins, since they are on different cycles.
// By sharing a pin, this frees up a precious IO pin for other purposes. In theory, all axes limit pins
// may be reduced to one pin, if all axes are homed with seperate cycles, or vice versa, all three axes

```

// on separate pin, but homed in one cycle. Also, it should be noted that the function of hard limits
// will not be affected by pin sharing.
// NOTE: Defaults are set for a traditional 3-axis CNC machine. Z-axis first to clear, followed by X & Y.

#if N_AXIS == 4 // 4 axis : homing

#define HOMING_CYCLE_0 (1<<AXIS_3) // Home Z axis first to clear workspace.

#define HOMING_CYCLE_1 ((1<<AXIS_1)|(1<<AXIS_2)) // OPTIONAL: uncomment to move X,Y at the same time.

//#define HOMING_CYCLE_1 (1<<AXIS_1) // Home X axis // OPTIONAL: uncomment to move only X at a time.

//#define HOMING_CYCLE_2 (1<<AXIS_2) // Home Y axis // OPTIONAL: uncomment to move only Y at a time.

//#define HOMING_CYCLE_3 (1<<AXIS_4) // Home 4th axis (A)

#elif N_AXIS == 5 // 5 axis : homing

#define HOMING_CYCLE_0 (1<<AXIS_3) // Home Z axis first to clear workspace.

#define HOMING_CYCLE_1 ((1<<AXIS_1)|(1<<AXIS_2)) // OPTIONAL: uncomment to move X,Y at the same time.

//#define HOMING_CYCLE_1 (1<<AXIS_1) // Home X axis // OPTIONAL: uncomment to move only X at a time.

//#define HOMING_CYCLE_2 (1<<AXIS_2) // Home Y axis // OPTIONAL: uncomment to move only Y at a time.

//#define HOMING_CYCLE_3 (1<<AXIS_4) // Home 4th axis (A)

//#define HOMING_CYCLE_4 (1<<AXIS_5) // Home 5th axis (B)

#elif N_AXIS == 6 // 6 axis : homing

#define HOMING_CYCLE_0 (1<<AXIS_3) // Home Z axis first to clear workspace.

#define HOMING_CYCLE_1 ((1<<AXIS_1)|(1<<AXIS_2)) // OPTIONAL: uncomment to move X,Y at the same time.

//#define HOMING_CYCLE_1 (1<<AXIS_1) // Home X axis // OPTIONAL: uncomment to move only X at a time.

//#define HOMING_CYCLE_2 (1<<AXIS_2) // Home Y axis // OPTIONAL: uncomment to move only Y at a time.

//#define HOMING_CYCLE_3 (1<<AXIS_4) // Home 4th axis (A)

//#define HOMING_CYCLE_4 (1<<AXIS_5) // Home 5th axis (B)

//#define HOMING_CYCLE_5 (1<<AXIS_6) // Home 6th axis (C)

#else // Classic 3 axis

#define HOMING_CYCLE_0 (1<<AXIS_3) // Home Z axis first to clear workspace.

#define HOMING_CYCLE_1 ((1<<AXIS_1)|(1<<AXIS_2)) // OPTIONAL: uncomment to move X,Y at the same time.

//#define HOMING_CYCLE_1 (1<<AXIS_1) // Home X axis // OPTIONAL: uncomment to move only X at a time.

//#define HOMING_CYCLE_2 (1<<AXIS_2) // Home Y axis // OPTIONAL: uncomment to move only Y at a time.

#endif

// NOTE: The following are two examples to setup homing for 2-axis machines.

// #define HOMING_CYCLE_0 ((1<<AXIS_1)|(1<<AXIS_2)) // NOT COMPATIBLE WITH COREXY: Homes both X-Y in one

// #define HOMING_CYCLE_0 (1<<AXIS_1) // COREXY COMPATIBLE: First home X

// #define HOMING_CYCLE_1 (1<<AXIS_2) // COREXY COMPATIBLE: Then home Y

// Number of homing cycles performed after when the machine initially jogs to limit switches.

// This help in preventing overshoot and should improve repeatability. This value should be one or

// greater.

#define N_HOMING_LOCATE_CYCLE 1 // Integer (1-128)

// Enables single axis homing commands. \$HX, \$HY, and \$HZ for X, Y, and Z-axis homing. The full homing
// cycle is still invoked by the \$H command. This is disabled by default. It's here only to address

// users that need to switch between a two-axis and three-axis machine. This is actually very rare.

// If you have a two-axis machine, DON'T USE THIS. Instead, just alter the homing cycle for two-axes.

#define HOMING_SINGLE_AXIS_COMMANDS // Default disabled. Uncomment to enable.

// After homing, Grbl will set by default the entire machine space into negative space, as is typical

// for professional CNC machines, regardless of where the limit switches are located. Uncomment this

// define to force Grbl to always set the machine origin at the homed location despite switch orientation.

```

// #define HOMING_FORCE_SET_ORIGIN // Uncomment to enable.

// Number of blocks Grbl executes upon startup. These blocks are stored in EEPROM, where the size
// and addresses are defined in settings.h. With the current settings, up to 2 startup blocks may
// be stored and executed in order. These startup blocks would typically be used to set the g-code
// parser state depending on user preferences.
#define N_STARTUP_LINE 2 // Integer (1-2)

// Number of floating decimal points printed by Grbl for certain value types. These settings are
// determined by realistic and commonly observed values in CNC machines. For example, position
// values cannot be less than 0.001mm or 0.0001in, because machines can not be physically more
// precise this. So, there is likely no need to change these, but you can if you need to here.
// NOTE: Must be an integer value from 0 to ~4. More than 4 may exhibit round-off errors.
#define N_DECIMAL_COORDVALUE_INCH 4 // Coordinate or position value in inches
#define N_DECIMAL_COORDVALUE_MM 3 // Coordinate or position value in mm
#define N_DECIMAL_RATEVALUE_INCH 1 // Rate or velocity value in in/min
#define N_DECIMAL_RATEVALUE_MM 0 // Rate or velocity value in mm/min
#define N_DECIMAL_SETTINGVALUE 3 // Decimals for floating point setting values
#define N_DECIMAL_RPMVALUE 0 // RPM value in rotations per min.

// If your machine has two limits switches wired in parallel to one axis, you will need to enable
// this feature. Since the two switches are sharing a single pin, there is no way for Grbl to tell
// which one is enabled. This option only effects homing, where if a limit is engaged, Grbl will
// alarm out and force the user to manually disengage the limit switch. Otherwise, if you have one
// limit switch for each axis, don't enable this option. By keeping it disabled, you can perform a
// homing cycle while on the limit switch and not have to move the machine off of it.
// #define LIMITS_TWO_SWITCHES_ON_AXES

// Upon a successful probe cycle, this option provides immediately feedback of the probe coordinates
// through an automatically generated message. If disabled, users can still access the last probe
// coordinates through Grbl '$#' print parameters.
#define MESSAGE_PROBE_COORDINATES // Enabled by default. Comment to disable.

// After the safety door switch has been toggled and restored, this setting sets the power-up delay
// between restoring the spindle and coolant and resuming the cycle.
#define SAFETY_DOOR_SPINDLE_DELAY 4.0 // Float (seconds)
#define SAFETY_DOOR_COOLANT_DELAY 1.0 // Float (seconds)

// Enable CoreXY kinematics. Use ONLY with CoreXY machines.
// IMPORTANT: If homing is enabled, you must reconfigure the homing cycle #defines above to
// #define HOMING_CYCLE_0 (1<<AXIS_1) and #define HOMING_CYCLE_1 (1<<AXIS_2)
// NOTE: This configuration option alters the motion of the X and Y axes to principle of operation
// defined at (http://corexy.com/theory.html). Motors are assumed to be positioned and wired exactly as
// described, if not, motions may move in strange directions. Grbl requires the CoreXY A and B motors
// have the same steps per mm internally.
// #define COREXY // Default disabled. Uncomment to enable.

// Inverts pin logic of the control command pins based on a mask. This essentially means you can use
// normally-closed switches on the specified pins, rather than the default normally-open switches.
// NOTE: The top option will mask and invert all control pins. The bottom option is an example of
// inverting only two control pins, the safety door and reset. See cpu_map.h for other bit definitions.

```

```

// #define INVERT_CONTROL_PIN_MASK CONTROL_MASK // Default disabled. Uncomment to disable.
// #define INVERT_CONTROL_PIN_MASK ((1<<CONTROL_SAFETY_DOOR_BIT)|(1<<CONTROL_RESET_BIT)) // Default disabled. Uncomment to enable.

// Inverts select limit pin states based on the following mask. This effects all limit pin functions,
// such as hard limits and homing. However, this is different from overall invert limits setting.
// This build option will invert only the limit pins defined here, and then the invert limits setting
// will be applied to all of them. This is useful when a user has a mixed set of limit pins with both
// normally-open(NO) and normally-closed(NC) switches installed on their machine.
// NOTE: PLEASE DO NOT USE THIS, unless you have a situation that needs it.
// #define INVERT_LIMIT_PIN_MASK ((1<<X_LIMIT_BIT)|(1<<Y_LIMIT_BIT)) // Default disabled. Uncomment to enable.

// Enable the following line to inverse logical behaviour (Normaly Open / Normaly Closed)
// of some min limit switches attached.
// #define INVERT_MIN_LIMIT_PIN_MASK ((1<<AXIS_1) | (1<<AXIS_2) | (1<<AXIS_3))
// Enable the following line to inverse logical behaviour (Normaly Open / Normaly Closed)
// of some max limit switches attached.
// #define INVERT_MAX_LIMIT_PIN_MASK ((1<<AXIS_1) | (1<<AXIS_2) | (1<<AXIS_3))

// Inverts the spindle enable pin from low-disabled/high-enabled to low-enabled/high-disabled. Useful
// for some pre-built electronic boards.
// #define INVERT_SPINDLE_ENABLE_PIN // Default disabled. Uncomment to enable.

// Inverts the selected coolant pin from low-disabled/high-enabled to low-enabled/high-disabled. Useful
// for some pre-built electronic boards.
// #define INVERT_COOLANT_FLOOD_PIN // Default disabled. Uncomment to enable.
// #define INVERT_COOLANT_MIST_PIN // Default disabled. Note: Enable M7 mist coolant in config.h

// Inverts the selected digital output pin from low-disabled/high-enabled to low-enabled/high-disabled.
// Useful for some pre-built electronic boards.
// #define INVERT_DIGITAL_OUTPUT_PIN_0 // Default disabled. Uncomment to enable.
// #define INVERT_DIGITAL_OUTPUT_PIN_1 // Default disabled. Uncomment to enable.
// #define INVERT_DIGITAL_OUTPUT_PIN_2 // Default disabled. Uncomment to enable.
// #define INVERT_DIGITAL_OUTPUT_PIN_3 // Default disabled. Uncomment to enable.

// Digital inputs: Uncomment the folloing line to enable the use of up to
// 4 digital input pins. Digital inputs work in the same way as the other input
// #define USE_DIGITAL_INPUT // Default disabled. Uncomment to enable.

// pins (probe, safety door, cycle start, reset, feed hold).
// Invert the digital input status. Default is normaly open switch between pin
// to GND, uncomment to use normaly closed switch.
// #define INVERT_DIGITAL_INPUT_PIN_0 // Default disabled. Uncomment to enable.
// #define INVERT_DIGITAL_INPUT_PIN_1 // Default disabled. Uncomment to enable.
// #define INVERT_DIGITAL_INPUT_PIN_2 // Default disabled. Uncomment to enable.
// #define INVERT_DIGITAL_INPUT_PIN_3 // Default disabled. Uncomment to enable.

// When Grbl powers-cycles or is hard reset with the Arduino reset button, Grbl boots up with no ALARM
// by default. This is to make it as simple as possible for new users to start using Grbl. When homing
// is enabled and a user has installed limit switches, Grbl will boot up in an ALARM state to indicate
// Grbl doesn't know its position and to force the user to home before proceeding. This option forces
// Grbl to always initialize into an ALARM state regardless of homing or not. This option is more for

```

```

// OEMs and LinuxCNC users that would like this power-cycle behavior.
// #define FORCE_INITIALIZATION_ALARM // Default disabled. Uncomment to enable.

// At power-up or a reset, Grbl will check the limit switch states to ensure they are not active
// before initialization. If it detects a problem and the hard limits setting is enabled, Grbl will
// simply message the user to check the limits and enter an alarm state, rather than idle. Grbl will
// not throw an alarm message.
#define CHECK_LIMITS_AT_INIT

// -----
// ADVANCED CONFIGURATION OPTIONS:

// Enables code for debugging purposes. Not for general use and always in constant flux.
// #define DEBUG // Uncomment to enable. Default disabled.

// Configure rapid, feed, and spindle override settings. These values define the max and min
// allowable override values and the coarse and fine increments per command received. Please
// note the allowable values in the descriptions following each define.
#define DEFAULT_FEED_OVERRIDE      100 // 100%. Don't change this value.
#define MAX_FEED_RATE_OVERRIDE     200 // Percent of programmed feed rate (100-255). Usually 120% or 200%
#define MIN_FEED_RATE_OVERRIDE     10 // Percent of programmed feed rate (1-100). Usually 50% or 1%
#define FEED_OVERRIDE_COARSE_INCREMENT  10 // (1-99). Usually 10%.
#define FEED_OVERRIDE_FINE_INCREMENT   1 // (1-99). Usually 1%.

#define DEFAULT_RAPID_OVERRIDE 100 // 100%. Don't change this value.
#define RAPID_OVERRIDE_MEDIUM  50 // Percent of rapid (1-99). Usually 50%.
#define RAPID_OVERRIDE_LOW     25 // Percent of rapid (1-99). Usually 25%.

#define DEFAULT_SPINDLE_SPEED_OVERRIDE 100 // 100%. Don't change this value.
#define MAX_SPINDLE_SPEED_OVERRIDE     200 // Percent of programmed spindle speed (100-255). Usually 200%.
#define MIN_SPINDLE_SPEED_OVERRIDE     10 // Percent of programmed spindle speed (1-100). Usually 10%.
#define SPINDLE_OVERRIDE_COARSE_INCREMENT 10 // (1-99). Usually 10%.
#define SPINDLE_OVERRIDE_FINE_INCREMENT   1 // (1-99). Usually 1%.

// When a M2 or M30 program end command is executed, most g-code states are restored to their defaults.
// This compile-time option includes the restoring of the feed, rapid, and spindle speed override values
// to their default values at program end.
#define RESTORE_OVERRIDES_AFTER_PROGRAM_END // Default enabled. Comment to disable.

// The status report change for Grbl v1.1 and after also removed the ability to disable/enable most data
// fields from the report. This caused issues for GUI developers, who've had to manage several scenarios
// and configurations. The increased efficiency of the new reporting style allows for all data fields to
// be sent without potential performance issues.
// NOTE: The options below are here only provide a way to disable certain data fields if a unique
// situation demands it, but be aware GUIs may depend on this data. If disabled, it may not be compatible.
#define REPORT_FIELD_BUFFER_STATE // Default enabled. Comment to disable.
#define REPORT_FIELD_PIN_STATE // Default enabled. Comment to disable.
#define REPORT_FIELD_CURRENT_FEED_SPEED // Default enabled. Comment to disable.
#define REPORT_FIELD_WORK_COORD_OFFSET // Default enabled. Comment to disable.
#define REPORT_FIELD_OVERRIDES // Default enabled. Comment to disable.
#define REPORT_FIELD_LINE_NUMBERS // Default enabled. Comment to disable.

```

```

// Some status report data isn't necessary for realtime, only intermittently, because the values don't
// change often. The following macros configures how many times a status report needs to be called before
// the associated data is refreshed and included in the status report. However, if one of these value
// changes, Grbl will automatically include this data in the next status report, regardless of what the
// count is at the time. This helps reduce the communication overhead involved with high frequency reporting
// and aggressive streaming. There is also a busy and an idle refresh count, which sets up Grbl to send
// refreshes more often when its not doing anything important. With a good GUI, this data doesn't need
// to be refreshed very often, on the order of a several seconds.
// NOTE: WCO refresh must be 2 or greater. OVR refresh must be 1 or greater.
#define REPORT_OVR_REFRESH_BUSY_COUNT 20 // (1-255)
#define REPORT_OVR_REFRESH_IDLE_COUNT 10 // (1-255) Must be less than or equal to the busy count
#define REPORT_WCO_REFRESH_BUSY_COUNT 30 // (2-255)
#define REPORT_WCO_REFRESH_IDLE_COUNT 10 // (2-255) Must be less than or equal to the busy count

// The temporal resolution of the acceleration management subsystem. A higher number gives smoother
// acceleration, particularly noticeable on machines that run at very high feedrates, but may negatively
// impact performance. The correct value for this parameter is machine dependent, so it's advised to
// set this only as high as needed. Approximate successful values can widely range from 50 to 200 or more.
// NOTE: Changing this value also changes the execution time of a segment in the step segment buffer.
// When increasing this value, this stores less overall time in the segment buffer and vice versa. Make
// certain the step segment buffer is increased/decreased to account for these changes.
#define ACCELERATION_TICKS_PER_SECOND 100

// Adaptive Multi-Axis Step Smoothing (AMASS) is an advanced feature that does what its name implies,
// smoothing the stepping of multi-axis motions. This feature smooths motion particularly at low step
// frequencies below 10kHz, where the aliasing between axes of multi-axis motions can cause audible
// noise and shake your machine. At even lower step frequencies, AMASS adapts and provides even better
// step smoothing. See stepper.c for more details on the AMASS system works.
#define ADAPTIVE_MULTI_AXIS_STEP_SMOOTHING // Default enabled. Comment to disable.

// Sets the maximum step rate allowed to be written as a Grbl setting. This option enables an error
// check in the settings module to prevent settings values that will exceed this limitation. The maximum
// step rate is strictly limited by the CPU speed and will change if something other than an AVR running
// at 16MHz is used.
// NOTE: For now disabled, will enable if flash space permits.
// #define MAX_STEP_RATE_HZ 30000 // Hz

// By default, Grbl sets all input pins to normal-high operation with their internal pull-up resistors
// enabled. This simplifies the wiring for users by requiring only a switch connected to ground,
// although its recommended that users take the extra step of wiring in low-pass filter to reduce
// electrical noise detected by the pin. If the user inverts the pin in Grbl settings, this just flips
// which high or low reading indicates an active signal. In normal operation, this means the user
// needs to connect a normal-open switch, but if inverted, this means the user should connect a
// normal-closed switch.
// The following options disable the internal pull-up resistors, sets the pins to a normal-low
// operation, and switches must be now connect to Vcc instead of ground. This also flips the meaning
// of the invert pin Grbl setting, where an inverted setting now means the user should connect a
// normal-open switch and vice versa.
// NOTE: All pins associated with the feature are disabled, i.e. XYZ limit pins, not individual axes.
// WARNING: When the pull-ups are disabled, this requires additional wiring with pull-down resistors!

```

```

#define DISABLE_LIMIT_PIN_PULL_UP
#define DISABLE_PROBE_PIN_PULL_UP
#define DISABLE_CONTROL_PIN_PULL_UP
#define DISABLE_DIGITAL_INPUT_PIN_PULL_UP

// Sets which axis the tool length offset is applied. Assumes the spindle is always parallel with
// the selected axis with the tool oriented toward the negative direction. In other words, a positive
// tool length offset value is subtracted from the current location.
#define TOOL_LENGTH_OFFSET_AXIS AXIS_3 // Default z-axis. Valid values are AXIS_1, AXIS_2, or AXIS_3.

// Used by variable spindle output only. This forces the PWM output to a minimum duty cycle when enabled.
// The PWM pin will still read 0V when the spindle is disabled. Most users will not need this option, but
// it may be useful in certain scenarios. This minimum PWM settings coincides with the spindle rpm minimum
// setting, like rpm max to max PWM. This is handy if you need a larger voltage difference between 0V disabled
// and the voltage set by the minimum PWM for minimum rpm. This difference is 0.02V per PWM value. So, when
// minimum PWM is at 1, only 0.02 volts separate enabled and disabled. At PWM 5, this would be 0.1V. Keep
// in mind that you will begin to lose PWM resolution with increased minimum PWM values, since you have less
// and less range over the total 255 PWM levels to signal different spindle speeds.
// NOTE: Compute duty cycle at the minimum PWM by this equation: (% duty cycle)=(SPINDLE_PWM_MIN_VALUE/255)*100
// #define SPINDLE_PWM_MIN_VALUE 5 // Default disabled. Uncomment to enable. Must be greater than zero. Integer (1-255)

// Alters the behavior of the spindle enable pin. By default, Grbl will not disable the enable pin if
// spindle speed is zero and M3/4 is active, but still sets the PWM output to zero. This allows the users
// to know if the spindle is active and use it as an additional control input. However, in some use cases,
// a user may want the enable pin to disable with a zero spindle speed and re-enable when spindle speed is
// greater than zero. This option does that.
// #define SPINDLE_ENABLE_OFF_WITH_ZERO_SPEED // Default disabled. Uncomment to enable.

// With this enabled, Grbl sends back an echo of the line it has received, which has been pre-parsed (spaces
// removed, capitalized letters, no comments) and is to be immediately executed by Grbl. Echoes will not be
// sent upon a line buffer overflow, but should for all normal lines sent to Grbl. For example, if a user
// sendss the line 'g1 x1.032 y2.45 (test comment)', Grbl will echo back in the form '[echo: G1X1.032Y2.45]'.
// NOTE: Only use this for debugging purposes!! When echoing, this takes up valuable resources and can effect
// performance. If absolutely needed for normal operation, the serial write buffer should be greatly increased
// to help minimize transmission waiting within the serial write protocol.
// #define REPORT_ECHO_LINE_RECEIVED // Default disabled. Uncomment to enable.

// Minimum planner junction speed. Sets the default minimum junction speed the planner plans to at
// every buffer block junction, except for starting from rest and end of the buffer, which are always
// zero. This value controls how fast the machine moves through junctions with no regard for acceleration
// limits or angle between neighboring block line move directions. This is useful for machines that can't
// tolerate the tool dwelling for a split second, i.e. 3d printers or laser cutters. If used, this value
// should not be much greater than zero or to the minimum value necessary for the machine to work.
#define MINIMUM_JUNCTION_SPEED 0.0 // (mm/min)

// Sets the minimum feed rate the planner will allow. Any value below it will be set to this minimum
// value. This also ensures that a planned motion always completes and accounts for any floating-point
// round-off errors. Although not recommended, a lower value than 1.0 mm/min will likely work in smaller
// machines, perhaps to 0.1mm/min, but your success may vary based on multiple factors.
#define MINIMUM_FEED_RATE 1.0 // (mm/min)

```

```

// Number of arc generation iterations by small angle approximation before exact arc trajectory
// correction with expensive sin() and cos() calculations. This parameter maybe decreased if there
// are issues with the accuracy of the arc generations, or increased if arc execution is getting
// bogged down by too many trig calculations.
#define N_ARC_CORRECTION 12 // Integer (1-255)

// The arc G2/3 g-code standard is problematic by definition. Radius-based arcs have horrible numerical
// errors when arc at semi-circles( $\pi$ ) or full-circles( $2\pi$ ). Offset-based arcs are much more accurate
// but still have a problem when arcs are full-circles ( $2\pi$ ). This define accounts for the floating
// point issues when offset-based arcs are commanded as full circles, but get interpreted as extremely
// small arcs with around machine epsilon ( $1.2e-7$ rad) due to numerical round-off and precision issues.
// This define value sets the machine epsilon cutoff to determine if the arc is a full-circle or not.
// NOTE: Be very careful when adjusting this value. It should always be greater than  $1.2e-7$  but not too
// much greater than this. The default setting should capture most, if not all, full arc error situations.
#define ARC_ANGULAR_TRAVEL_EPSILON 5E-7 // Float (radians)

// Time delay increments performed during a dwell. The default value is set at 50ms, which provides
// a maximum time delay of roughly 55 minutes, more than enough for most any application. Increasing
// this delay will increase the maximum dwell time linearly, but also reduces the responsiveness of
// run-time command executions, like status reports, since these are performed between each dwell
// time step. Also, keep in mind that the Arduino delay timer is not very accurate for long delays.
#define DWELL_TIME_STEP 50 // Integer (1-255) (milliseconds)

// Creates a delay between the direction pin setting and corresponding step pulse by creating
// another interrupt (Timer2 compare) to manage it. The main Grbl interrupt (Timer1 compare)
// sets the direction pins, and does not immediately set the stepper pins, as it would in
// normal operation. The Timer2 compare fires next to set the stepper pins after the step
// pulse delay time, and Timer2 overflow will complete the step pulse, except now delayed
// by the step pulse time plus the step pulse delay. (Thanks langwadt for the idea!)
// NOTE: Uncomment to enable. The recommended delay must be  $> 3\mu s$ , and, when added with the
// user-supplied step pulse time, the total time must not exceed  $127\mu s$ . Reported successful
// values for certain setups have ranged from 5 to  $20\mu s$ .
// #define STEP_PULSE_DELAY 10 // Step pulse delay in microseconds. Default disabled.

// The number of linear motions in the planner buffer to be planned at any give time. The vast
// majority of RAM that Grbl uses is based on this buffer size. Only increase if there is extra
// available RAM, like when re-compiling for a Mega or Sanguino. Or decrease if the Arduino
// begins to crash due to the lack of available RAM or if the CPU is having trouble keeping
// up with planning new incoming motions as they are executed.
// #define BLOCK_BUFFER_SIZE 36 // Uncomment to override default in planner.h.

// Governs the size of the intermediary step segment buffer between the step execution algorithm
// and the planner blocks. Each segment is set of steps executed at a constant velocity over a
// fixed time defined by ACCELERATION_TICKS_PER_SECOND. They are computed such that the planner
// block velocity profile is traced exactly. The size of this buffer governs how much step
// execution lead time there is for other Grbl processes have to compute and do their thing
// before having to come back and refill this buffer, currently at  $\sim 50$ msec of step moves.
// #define SEGMENT_BUFFER_SIZE 10 // Uncomment to override default in stepper.h.

// Line buffer size from the serial input stream to be executed. Also, governs the size of
// each of the startup blocks, as they are each stored as a string of this size. Make sure

```

```

// to account for the available EEPROM at the defined memory address in settings.h and for
// the number of desired startup blocks.
// NOTE: 80 characters is not a problem except for extreme cases, but the line buffer size
// can be too small and g-code blocks can get truncated. Officially, the g-code standards
// support up to 256 characters. In future versions, this default will be increased, when
// we know how much extra memory space we can re-invest into this.
// #define LINE_BUFFER_SIZE 256 // Uncomment to override default in protocol.h

// Serial send and receive buffer size. The receive buffer is often used as another streaming
// buffer to store incoming blocks to be processed by Grbl when its ready. Most streaming
// interfaces will character count and track each block send to each block response. So,
// increase the receive buffer if a deeper receive buffer is needed for streaming and available
// memory allows. The send buffer primarily handles messages in Grbl. Only increase if large
// messages are sent and Grbl begins to stall, waiting to send the rest of the message.
// NOTE: Buffer size values must be greater than zero and less than 256.
// #define RX_BUFFER_SIZE 255 // Uncomment to override defaults in serial.h
// #define TX_BUFFER_SIZE 255

// The maximum line length of a data string stored in EEPROM. Used by startup lines and build
// info. This size differs from the LINE_BUFFER_SIZE as the EEPROM is usually limited in size.
// NOTE: Be very careful when changing this value. Check EEPROM address locations to make sure
// these string storage locations won't corrupt one another.
// #define EEPROM_LINE_SIZE 80 // Uncomment to override defaults in settings.h

// Configures the position after a probing cycle during Grbl's check mode. Disabled sets
// the position to the probe target, when enabled sets the position to the start position.
// #define SET_CHECK_MODE_PROBE_TO_START // Default disabled. Uncomment to enable.

// Force Grbl to check the state of the hard limit switches when the processor detects a pin
// change inside the hard limit ISR routine. By default, Grbl will trigger the hard limits
// alarm upon any pin change, since bouncing switches can cause a state check like this to
// misread the pin. When hard limits are triggered, they should be 100% reliable, which is the
// reason that this option is disabled by default. Only if your system/electronics can guarantee
// that the switches don't bounce, we recommend enabling this option. This will help prevent
// triggering a hard limit when the machine disengages from the switch.
// NOTE: This option has no effect if SOFTWARE_DEBOUNCE is enabled.
// #define HARD_LIMIT_FORCE_STATE_CHECK // Default disabled. Uncomment to enable.

// Adjusts homing cycle search and locate scalars. These are the multipliers used by Grbl's
// homing cycle to ensure the limit switches are engaged and cleared through each phase of
// the cycle. The search phase uses the axes max-travel setting times the SEARCH_SCALAR to
// determine distance to look for the limit switch. Once found, the locate phase begins and
// uses the homing pull-off distance setting times the LOCATE_SCALAR to pull-off and re-engage
// the limit switch.
// NOTE: Both of these values must be greater than 1.0 to ensure proper function.
// #define HOMING_AXIS_SEARCH_SCALAR 1.5 // Uncomment to override defaults in limits.c.
// #define HOMING_AXIS_LOCATE_SCALAR 10.0 // Uncomment to override defaults in limits.c.

// Enable the '$RST=*', '$RST=$', and '$RST=#' eeprom restore commands. There are cases where
// these commands may be undesirable. Simply comment the desired macro to disable it.
// NOTE: See SETTINGS_RESTORE_ALL macro for customizing the '$RST=*' command.

```

```

#define ENABLE_RESTORE_EEPROM_WIPE_ALL // '$RST=*' Default enabled. Comment to disable.
#define ENABLE_RESTORE_EEPROM_DEFAULT_SETTINGS // '$RST=$' Default enabled. Comment to disable.
#define ENABLE_RESTORE_EEPROM_CLEAR_PARAMETERS // '$RST=#' Default enabled. Comment to disable.

// Defines the EEPROM data restored upon a settings version change and `RST=*` command. Whenever the
// the settings or other EEPROM data structure changes between Grbl versions, Grbl will automatically
// wipe and restore the EEPROM. This macro controls what data is wiped and restored. This is useful
// particularly for OEMs that need to retain certain data. For example, the BUILD_INFO string can be
// written into the Arduino EEPROM via a separate .INO sketch to contain product data. Altering this
// macro to not restore the build info EEPROM will ensure this data is retained after firmware upgrades.
// NOTE: Uncomment to override defaults in settings.h
// #define SETTINGS_RESTORE_ALL (SETTINGS_RESTORE_DEFAULTS | SETTINGS_RESTORE_PARAMETERS | SE

// Enable the '$I=(string)' build info write command. If disabled, any existing build info data must
// be placed into EEPROM via external means with a valid checksum value. This macro option is useful
// to prevent this data from being over-written by a user, when used to store OEM product data.
// NOTE: If disabled and to ensure Grbl can never alter the build info line, you'll also need to enable
// the SETTING_RESTORE_ALL macro above and remove SETTINGS_RESTORE_BUILD_INFO from the mask.
// NOTE: See the included grblWrite_BuildInfo.ino example file to write this string separately.
#define ENABLE_BUILD_INFO_WRITE_COMMAND // '$I=' Default enabled. Comment to disable.

// AVR processors require all interrupts to be disabled during an EEPROM write. This includes both
// the stepper ISRs and serial comm ISRs. In the event of a long EEPROM write, this ISR pause can
// cause active stepping to lose position and serial receive data to be lost. This configuration
// option forces the planner buffer to completely empty whenever the EEPROM is written to prevent
// any chance of lost steps.
// However, this doesn't prevent issues with lost serial RX data during an EEPROM write, especially
// if a GUI is preemptively filling up the serial RX buffer simultaneously. It's highly advised for
// GUIs to flag these gcodes (G10,G28.1,G30.1) to always wait for an 'ok' after a block containing
// one of these commands before sending more data to eliminate this issue.
// NOTE: Most EEPROM write commands are implicitly blocked during a job (all '$' commands). However,
// coordinate set g-code commands (G10,G28/30.1) are not, since they are part of an active streaming
// job. At this time, this option only forces a planner buffer sync with these g-code commands.
#define FORCE_BUFFER_SYNC_DURING_EEPROM_WRITE // Default enabled. Comment to disable.

// In Grbl v0.9 and prior, there is an old outstanding bug where the `WPos:` work position reported
// may not correlate to what is executing, because `WPos:` is based on the g-code parser state, which
// can be several motions behind. This option forces the planner buffer to empty, sync, and stop
// motion whenever there is a command that alters the work coordinate offsets `G10,G43.1,G92,G54-59`.
// This is the simplest way to ensure `WPos:` is always correct. Fortunately, it's exceedingly rare
// that any of these commands are used need continuous motions through them.
#define FORCE_BUFFER_SYNC_DURING_WCO_CHANGE // Default enabled. Comment to disable.

// By default, Grbl disables feed rate overrides for all G38.x probe cycle commands. Although this
// may be different than some pro-class machine control, it's arguable that it should be this way.
// Most probe sensors produce different levels of error that is dependent on rate of speed. By
// keeping probing cycles to their programmed feed rates, the probe sensor should be a lot more
// repeatable. If needed, you can disable this behavior by uncommenting the define below.
// #define ALLOW_FEED_OVERRIDE_DURING_PROBE_CYCLES // Default disabled. Uncomment to enable.

// Enables and configures parking motion methods upon a safety door state. Primarily for OEMs

```

```

// that desire this feature for their integrated machines. At the moment, Grbl assumes that
// the parking motion only involves one axis, although the parking implementation was written
// to be easily refactored for any number of motions on different axes by altering the parking
// source code. At this time, Grbl only supports parking one axis (typically the Z-axis) that
// moves in the positive direction upon retracting and negative direction upon restoring position.
// The motion executes with a slow pull-out retraction motion, power-down, and a fast park.
// Restoring to the resume position follows these set motions in reverse: fast restore to
// pull-out position, power-up with a time-out, and plunge back to the original position at the
// slower pull-out rate.
// NOTE: Still a work-in-progress. Machine coordinates must be in all negative space and
// does not work with HOMING_FORCE_SET_ORIGIN enabled. Parking motion also moves only in
// positive direction.
// #define PARKING_ENABLE // Default disabled. Uncomment to enable

// Configure options for the parking motion, if enabled.
#define PARKING_AXIS AXIS_3 // Define which axis that performs the parking motion
#define PARKING_TARGET -5.0 // Parking axis target. In mm, as machine coordinate [-max_travel,0].
#define PARKING_RATE 500.0 // Parking fast rate after pull-out in mm/min.
#define PARKING_PULLOUT_RATE 100.0 // Pull-out/plunge slow feed rate in mm/min.
#define PARKING_PULLOUT_INCREMENT 5.0 // Spindle pull-out and plunge distance in mm. Incremental distance.
    // Must be positive value or equal to zero.

// Enables a special set of M-code commands that enables and disables the parking motion.
// These are controlled by `M56`, `M56 P1`, or `M56 Px` to enable and `M56 P0` to disable.
// The command is modal and will be set after a planner sync. Since it is g-code, it is
// executed in sync with g-code commands. It is not a real-time command.
// NOTE: PARKING_ENABLE is required. By default, M56 is active upon initialization. Use
// DEACTIVATE_PARKING_UPON_INIT to set M56 P0 as the power-up default.
// #define ENABLE_PARKING_OVERRIDE_CONTROL // Default disabled. Uncomment to enable
// #define DEACTIVATE_PARKING_UPON_INIT // Default disabled. Uncomment to enable.

// This option will automatically disab
// Enables and configures Grbl's sleep mode feature. If the spindle or coolant are powered and Grbl
// is not actively moving or receiving any commands, a sleep timer will start. If any data or commands
// are received, the sleep timer will reset and restart until the above condition are not satisfied.
// If the sleep timer elapses, Grbl will immediately execute the sleep mode by shutting down the spindle
// and coolant and entering a safe sleep state. If parking is enabled, Grbl will park the machine as
// well. While in sleep mode, only a hard/soft reset will exit it and the job will be unrecoverable.
// NOTE: Sleep mode is a safety feature, primarily to address communication disconnect problems. To
// keep Grbl from sleeping, employ a stream of '?' status report commands as a connection "heartbeat".
// #define SLEEP_ENABLE // Default disabled. Uncomment to enable.
#define SLEEP_DURATION 5.0 // Float (0.25 - 61.0) seconds before sleep mode is executed.

// This option will automatically disable the laser during a feed hold by invoking a spindle stop
// override immediately after coming to a stop. However, this also means that the laser still may
// be reenabled by disabling the spindle stop override, if needed. This is purely a safety feature
// to ensure the laser doesn't inadvertently remain powered while at a stop and cause a fire.
#define DISABLE_LASER_DURING_HOLD // Default enabled. Comment to disable.

// Enables a piecewise linear model of the spindle PWM/speed output. Requires a solution by the
// 'fit_nonlinear_spindle.py' script in the /doc/script folder of the repo. See file comments

```

```

// on how to gather spindle data and run the script to generate a solution.
// #define ENABLE_PIECEWISE_LINEAR_SPINDLE // Default disabled. Uncomment to enable.
// ENABLE_PIECEWISE_LINEAR_SPINDLE is not compatible with the SEPARATE_SPINDLE_LASER_PIN option
#ifdef ENABLE_PIECEWISE_LINEAR_SPINDLE
    #ifdef SEPARATE_SPINDLE_LASER_PIN
        #error ENABLE_PIECEWISE_LINEAR_SPINDLE compile option is not compatible with the SEPARATE_SPINDLE_LASER_PIN option
    #endif
#endif

// N_PIECES, RPM_MAX, RPM_MIN, RPM_POINTxx, and RPM_LINE_XX constants are all set and given by
// the 'fit_nonlinear_spindle.py' script solution. Used only when ENABLE_PIECEWISE_LINEAR_SPINDLE
// is enabled. Make sure the constant values are exactly the same as the script solution.
// NOTE: When N_PIECES < 4, unused RPM_LINE and RPM_POINT defines are not required and omitted.
#define N_PIECES 4 // Integer (1-4). Number of piecewise lines used in script solution.
#define RPM_MAX 11686.4 // Max RPM of model. $30 > RPM_MAX will be limited to RPM_MAX.
#define RPM_MIN 202.5 // Min RPM of model. $31 < RPM_MIN will be limited to RPM_MIN.
#define RPM_POINT12 6145.4 // Used N_PIECES >=2. Junction point between lines 1 and 2.
#define RPM_POINT23 9627.8 // Used N_PIECES >=3. Junction point between lines 2 and 3.
#define RPM_POINT34 10813.9 // Used N_PIECES = 4. Junction point between lines 3 and 4.
#define RPM_LINE_A1 3.197101e-03 // Used N_PIECES >=1. A and B constants of line 1.
#define RPM_LINE_B1 -3.526076e-1
#define RPM_LINE_A2 1.722950e-2 // Used N_PIECES >=2. A and B constants of line 2.
#define RPM_LINE_B2 8.588176e+01
#define RPM_LINE_A3 5.901518e-02 // Used N_PIECES >=3. A and B constants of line 3.
#define RPM_LINE_B3 4.881851e+02
#define RPM_LINE_A4 1.203413e-01 // Used N_PIECES = 4. A and B constants of line 4.
#define RPM_LINE_B4 1.151360e+03

/* -----
OEM Single File Configuration Option

Instructions: Paste the cpu_map and default setting definitions below without an enclosing
#ifdef. Comment out the CPU_MAP_xxx and DEFAULT_xxx defines at the top of this file, and
the compiler will ignore the contents of defaults.h and cpu_map.h and use the definitions
below.
*/

// Paste CPU_MAP definitions here.

// Paste default settings definitions here.

#endif

```

File: cpu_map.h

Directory: C:/Users/sense/Downloads/grbl-Mega-5X-edge/grbl-Mega-5X-edge/grbl

/*

cpu_map.h - CPU and pin mapping configuration file
Part of Grbl

Copyright (c) 2017-2022 Gauthier Briere

Copyright (c) 2012-2016 Sungeun K. Jeon for Gnea Research LLC

Grbl is free software: you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or
(at your option) any later version.

Grbl is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

*/

/* The cpu_map.h files serve as a central pin mapping selection file for different
processor types or alternative pin layouts. This version of Grbl supports only the
Arduino Mega2560. */

```
#ifndef cpu_map_h
#define cpu_map_h
```

```
#ifdef CPU_MAP_2560_RAMPS_BOARD // (Arduino Mega 2560) with Ramps 1.4 Board
#include "nuts_bolts.h"
```

```
// Serial port interrupt vectors
#define SERIAL_RX USART0_RX_vect
#define SERIAL_UDRE USART0_UDRE_vect
```

```
// Define ports and pins
#define DDR(port) DDR##port
#define _DDR(port) DDR(port)
#define PORT(port) PORT##port
#define _PORT(port) PORT(port)
#define PIN(pin) PIN##pin
#define _PIN(pin) PIN(pin)
```

```
// --- Y2 / A axis (E0 driver) -----
#define STEP_PIN_A 26 // E0 STEP
#define DIR_PIN_A 28 // E0 DIR
#define ENABLE_PIN_A 24 // E0 ENABLE
//-----
```

```

// Define step pulse output pins.

#define STEP_PORT_0 F
#define STEP_PORT_1 F
#define STEP_PORT_2 L
#if N_AXIS > 3
    #define STEP_PORT_3 A // Axis number 4 (Ramps E0)
#endif
#if N_AXIS > 4
    #define STEP_PORT_4 C // Axis number 5 (Ramps E1)
#endif
#if N_AXIS > 5
    #define STEP_PORT_5 L // Axis number 6 (Ramps Aux-3 D49)
#endif
#define STEP_BIT_0 0 // X Step - Pin A0
#define STEP_BIT_1 6 // Y Step - Pin A6
#define STEP_BIT_2 3 // Z Step - Pin D46
#if N_AXIS > 3
    #define STEP_BIT_3 4 // Axis number 4 Step - Pin D26
#endif
#if N_AXIS > 4
    #define STEP_BIT_4 1 // Axis number 5 Step - Pin D36
#endif
#if N_AXIS > 5
    #define STEP_BIT_5 0 // Axis number 6 Step - Pin D49
#endif
#define _STEP_BIT(i) STEP_BIT_##i
#define STEP_BIT(i) _STEP_BIT(i)
#define STEP_DDR(i) _DDR(STEP_PORT_##i)
#define _STEP_PORT(i) _PORT(STEP_PORT_##i)
#define STEP_PORT(i) _STEP_PORT(i)
#define STEP_PIN(i) _PIN(STEP_PORT_##i)

// Define step direction output pins.
#define DIRECTION_PORT_0 F
#define DIRECTION_PORT_1 F
#define DIRECTION_PORT_2 L
#if N_AXIS > 3
    #define DIRECTION_PORT_3 A // Axis number 4 (Ramps E0)
#endif
#if N_AXIS > 4
    #define DIRECTION_PORT_4 C // Axis number 5 (Ramps E1)
#endif
#if N_AXIS > 5
    #define DIRECTION_PORT_5 B // Axis number 6 (Ramps Aux-3 D51)
#endif
#define DIRECTION_BIT_0 1 // X Dir - Pin A1
#define DIRECTION_BIT_1 7 // Y Dir - Pin A7
#define DIRECTION_BIT_2 1 // Z Dir - Pin D48
#if N_AXIS > 3

```

```

#define DIRECTION_BIT_3 6 // Axis number 4 Step - Pin D28
#endif
#if N_AXIS > 4
#define DIRECTION_BIT_4 3 // Axis number 5 Step - Pin D34
#endif
#if N_AXIS > 5
#define DIRECTION_BIT_5 2 // Axis number 6 Step - Pin D51
#endif
#define _DIRECTION_BIT(i) DIRECTION_BIT_##i
#define DIRECTION_BIT(i) _DIRECTION_BIT(i)
#define DIRECTION_DDR(i) _DDR(DIRECTION_PORT_##i)
#define _DIRECTION_PORT(i) _PORT(DIRECTION_PORT_##i)
#define DIRECTION_PORT(i) _DIRECTION_PORT(i)
#define DIRECTION_PIN(i) _PIN(DIRECTION_PORT_##i)

// Define stepper driver enable/disable output pin.
#define STEPPER_DISABLE_PORT_0 D
#define STEPPER_DISABLE_PORT_1 F
#define STEPPER_DISABLE_PORT_2 K
#if N_AXIS > 3
#define STEPPER_DISABLE_PORT_3 A // Axis number 4 (Ramps E0)
#endif
#if N_AXIS > 4
#define STEPPER_DISABLE_PORT_4 C // Axis number 5 (Ramps E1)
#endif
#if N_AXIS > 5
#define STEPPER_DISABLE_PORT_5 B // Axis number 5 (Ramps Aux-3 D53)
#endif
#define STEPPER_DISABLE_BIT_0 7 // X Enable - Pin D38
#define STEPPER_DISABLE_BIT_1 2 // Y Enable - Pin A2
#define STEPPER_DISABLE_BIT_2 0 // Z Enable - Pin A8
#if N_AXIS > 3
#define STEPPER_DISABLE_BIT_3 2 // Axis number 4 Step - Pin D24
#endif
#if N_AXIS > 4
#define STEPPER_DISABLE_BIT_4 7 // Axis number 5 Step - Pin D30
#endif
#if N_AXIS > 5
#define STEPPER_DISABLE_BIT_5 0 // Axis number 5 Step - Pin D53
#endif
#define STEPPER_DISABLE_BIT(i) STEPPER_DISABLE_BIT_##i
#define STEPPER_DISABLE_DDR(i) _DDR(STEPPER_DISABLE_PORT_##i)
#define STEPPER_DISABLE_PORT(i) _PORT(STEPPER_DISABLE_PORT_##i)
#define STEPPER_DISABLE_PIN(i) _PIN(STEPPER_DISABLE_PORT_##i)

// Define homing/hard limit switch input pins and limit interrupt vectors.
#define MIN_LIMIT_PORT_0 E
#define MIN_LIMIT_PORT_1 J
#define MIN_LIMIT_PORT_2 D
#if N_AXIS > 3
#define MIN_LIMIT_PORT_3 L

```

```

#endif
#if N_AXIS > 4
    #define MIN_LIMIT_PORT_4 L
#endif
#if N_AXIS > 5
    #define MIN_LIMIT_PORT_5 F // (Ramps Aux-1 D57)
#endif
#define MIN_LIMIT_BIT_0 5 // X Limit Min - Pin D3
#define MIN_LIMIT_BIT_1 1 // Y Limit Min - Pin D14
#define MIN_LIMIT_BIT_2 3 // Z Limit Min - Pin D18
#if N_AXIS > 3
    #define MIN_LIMIT_BIT_3 7 // Axis number 4 : RAMPS AUX2 pin D42
#endif
#if N_AXIS > 4
    #define MIN_LIMIT_BIT_4 5 // Axis number 5 : RAMPS AUX2 pin D44
#endif
#if N_AXIS > 5
    #define MIN_LIMIT_BIT_5 3 // Axis number 6 : RAMPS AUX2 pin D57
#endif
#define _MIN_LIMIT_BIT(i) MIN_LIMIT_BIT_##i
#define MIN_LIMIT_BIT(i) _MIN_LIMIT_BIT(i)
#define MIN_LIMIT_DDR(i) _DDR(MIN_LIMIT_PORT_##i)
#define MIN_LIMIT_PORT(i) _PORT(MIN_LIMIT_PORT_##i)
#define MIN_LIMIT_PIN(i) _PIN(MIN_LIMIT_PORT_##i)

#define MAX_LIMIT_PORT_0 E
#define MAX_LIMIT_PORT_1 J
#define MAX_LIMIT_PORT_2 D
#if N_AXIS > 3
    #define MAX_LIMIT_PORT_3 G
#endif
#if N_AXIS > 4
    #define MAX_LIMIT_PORT_4 F
#endif
#if N_AXIS > 5
    #define MAX_LIMIT_PORT_5 F // (Ramps Aux-3 D58)
#endif
#define MAX_LIMIT_BIT_0 4 // X Limit Max - Pin D2
#define MAX_LIMIT_BIT_1 0 // Y Limit Max - Pin D15
#define MAX_LIMIT_BIT_2 2 // Z Limit Max - Pin D19
#if N_AXIS > 3
    #define MAX_LIMIT_BIT_3 1 // Axis number 4 : RAMPS AUX2 pin D40
#endif
#if N_AXIS > 4
    #define MAX_LIMIT_BIT_4 5 // Axis number 5 : RAMPS AUX2 pin D59
#endif
#if N_AXIS > 5
    #define MAX_LIMIT_BIT_5 4 // Axis number 6 : RAMPS AUX2 pin D58
#endif
#define _MAX_LIMIT_BIT(i) MAX_LIMIT_BIT_##i
#define MAX_LIMIT_BIT(i) _MAX_LIMIT_BIT(i)

```

```

#define MAX_LIMIT_DDR(i) _DDR(MAX_LIMIT_PORT_##i)
#define MAX_LIMIT_PORT(i) _PORT(MAX_LIMIT_PORT_##i)
#define MAX_LIMIT_PIN(i) _PIN(MAX_LIMIT_PORT_##i)

// Enable Hardware limit support for RAMPS without using interrupt...
// Warning! bouncing switches can cause a state check like this to misread the pin.
// When hard limits are triggered, they should be 100% reliable.
// The RAMPS_HW_LIMIT is implemented inside the stepper driver interrupt. Depending of your
// hardware, this can affect the max speed possibility of movments
// Disabled by default for performance optimization, uncomment to enable.
// #define ENABLE_RAMPS_HW_LIMITS

// Define spindle enable and spindle direction output pins.
#define SPINDLE_ENABLE_DDR    DDRG
#define SPINDLE_ENABLE_PORT   PORTG
#define SPINDLE_ENABLE_BIT    5 // MEGA2560 Digital Pin 4 - Ramps 1.4 Servo 4 Signal pin (D4)
#define SPINDLE_DIRECTION_DDR DDRE
#define SPINDLE_DIRECTION_PORT PORTE
#define SPINDLE_DIRECTION_BIT 3 // MEGA2560 Digital Pin 5 - Ramps 1.4 Servo 3 Signal pin (D5)

// Define flood and mist coolant enable output pins.
#define COOLANT_FLOOD_DDR    DDRB
#define COOLANT_FLOOD_PORT   PORTB
#define COOLANT_FLOOD_BIT    4 // MEGA2560 Digital Pin 10 - Ramps 1.4 12v output
#define COOLANT_MIST_DDR     DDRH
#define COOLANT_MIST_PORT    PORTH
#define COOLANT_MIST_BIT     6 // MEGA2560 Digital Pin 9 - Ramps 1.4 12v output

// Define M62 - M65 Digital Output Control ports
// D16 D17 D23 D25
#define DIGITAL_OUTPUT_DDR_0 DDRH
#define DIGITAL_OUTPUT_PORT_0 PORTH
#define DIGITAL_OUTPUT_BIT_0 1 // MEGA2560 Digital Pin 16 - Ramps 1.4 AUX-4 D16
#define DIGITAL_OUTPUT_DDR_1 DDRH
#define DIGITAL_OUTPUT_PORT_1 PORTH
#define DIGITAL_OUTPUT_BIT_1 0 // MEGA2560 Digital Pin 17 - Ramps 1.4 AUX-4 D17
#define DIGITAL_OUTPUT_DDR_2 DDRA
#define DIGITAL_OUTPUT_PORT_2 PORTA
#define DIGITAL_OUTPUT_BIT_2 1 // MEGA2560 Digital Pin 23 - Ramps 1.4 AUX-4 D23
#define DIGITAL_OUTPUT_DDR_3 DDRA
#define DIGITAL_OUTPUT_PORT_3 PORTA
#define DIGITAL_OUTPUT_BIT_3 3 // MEGA2560 Digital Pin 23 - Ramps 1.4 AUX-4 D25

// Define user-control CONTROLS (cycle start, reset, feed hold) input pins.
// NOTE: All CONTROLS pins must be on the same port and not on a port with other input pins (limits).
#define CONTROL_DDR    DDRK
#define CONTROL_PIN     PINK
#define CONTROL_PORT    PORTK
#define CONTROL_RESET_BIT    1 // Pin A9 - RAMPS Aux 2 Port
#define CONTROL_FEED_HOLD_BIT 2 // Pin A10 - RAMPS Aux 2 Port
#define CONTROL_CYCLE_START_BIT 3 // Pin A11 - RAMPS Aux 2 Port

```

```

#define CONTROL_SAFETY_DOOR_BIT 4 // Pin A12 - RAMPS Aux 2 Port
#define CONTROL_INT    PCIE2 // Pin change interrupt enable pin
#define CONTROL_INT_vect PCINT2_vect
#define CONTROL_PCMSK    PCMSK2 // Pin change interrupt register
#define CONTROL_MASK    ((1<<CONTROL_RESET_BIT)|(1<<CONTROL_FEED_HOLD_BIT)|(1<<CONTROL_CYCLETIME_BIT))

// Define probe switch input pin.
#define PROBE_DDR    DDRK
#define PROBE_PIN    PINK
#define PROBE_PORT    PORTK
#define PROBE_BIT    7 // MEGA2560 Analog Pin 15
#define PROBE_MASK    (1<<PROBE_BIT)

#ifdef USE_ANALOG_INPUT
    // Define Analog input
    #define ANALOG_INPUT_DDR_0    DDRK
    #define ANALOG_INPUT_PIN_0    PINK
    #define ANALOG_INPUT_PORT_0    PORTK
    #define ANALOG_INPUT_BIT_0    6 // MEGA2560 Analog Pin 14
    #define ANALOG_INPUT_MASK_0    (1<<ANALOG_INPUT_BIT_0)
    #define ANALOG_INPUT_DDR_1    DDRK
    #define ANALOG_INPUT_PIN_1    PINK
    #define ANALOG_INPUT_PORT_1    PORTK
    #define ANALOG_INPUT_BIT_1    5 // MEGA2560 Analog Pin 13
    #define ANALOG_INPUT_MASK_1    (1<<ANALOG_INPUT_BIT_1)
#endif

#ifdef USE_DIGITAL_INPUT
    // Define digital input
    #define DIGITAL_INPUT_DDR_0    DDRA
    #define DIGITAL_INPUT_PIN_0    PINA
    #define DIGITAL_INPUT_PORT_0    PORTA
    #define DIGITAL_INPUT_BIT_0    5 // MEGA2560 Port D27
    #define DIGITAL_INPUT_MASK_0    (1<<DIGITAL_INPUT_BIT_0)
    #define DIGITAL_INPUT_DDR_1    DDRA
    #define DIGITAL_INPUT_PIN_1    PINA
    #define DIGITAL_INPUT_PORT_1    PORTA
    #define DIGITAL_INPUT_BIT_1    7 // MEGA2560 Port D29
    #define DIGITAL_INPUT_MASK_1    (1<<DIGITAL_INPUT_BIT_1)
    #define DIGITAL_INPUT_DDR_2    DDRC
    #define DIGITAL_INPUT_PIN_2    PINC
    #define DIGITAL_INPUT_PORT_2    PORTC
    #define DIGITAL_INPUT_BIT_2    6 // MEGA2560 Port D31
    #define DIGITAL_INPUT_MASK_2    (1<<DIGITAL_INPUT_BIT_2)
    #define DIGITAL_INPUT_DDR_3    DDRC
    #define DIGITAL_INPUT_PIN_3    PINC
    #define DIGITAL_INPUT_PORT_3    PORTC
    #define DIGITAL_INPUT_BIT_3    4 // MEGA2560 Port D33
    #define DIGITAL_INPUT_MASK_3    (1<<DIGITAL_INPUT_BIT_3)
#endif

```

```
//-----
// Advanced Configuration Below You should not need to touch these variables
//-----

// Spindle PWM configuration :
// list of timers in Arduino Mega 2560
// TIMER0 (controls pin D13, D4);    => Timer0 is used by stepper.c
// TIMER1 (controls pin D12, D11);   => Timer1 is used by stepper.c
// TIMER2 (controls pin D10, D9);    => Timer2 is used by analog output or spindle PWM on D9
// TIMER3 (controls pin D5, D3, D2); => Timer3 is used by sleep.c
// TIMER4 (controls pin D8, D7, D6); => Timer4 is used by analog output or spindle PWM on D8 or D6
// TIMER5 (controls pin D46, D45, D44); => Timer5 is unused by grbl-Mega-5X. It's possible to add
//                                     PWM capability to ports D44 (RAMPS AUX-2), D45 (RAMPS AUX-4).
//                                     D46 is not available for PWM because it's used by Z step.
// Arduino pin number and the corresponding register for controlling the duty cycle :
// Pin  Register
// 2  OCR3B
// 3  OCR3C
// 4  OCR4C
// 5  OCR3A
// 6  OCR4A
// 7  OCR4B
// 8  OCR4C
// 9  OCR2B
// 10 OCR2A
// 11 OCR1A
// 12 OCR1B
// 13 OCR0A
// 44 OCR5C
// 45 OCR5B
// 46 OCR5A

#if defined(SPINDLE_PWM_ON_D8)

// Set Timer up to use TIMER4B which is attached to Digital Pin 8 - Ramps 1.4 12v output with heat sink
#define SPINDLE_PWM_MAX_VALUE 1024.0 // Translates to about 1.9 kHz PWM frequency at 1/8 prescaler
#ifndef SPINDLE_PWM_MIN_VALUE
#define SPINDLE_PWM_MIN_VALUE 1 // Must be greater than zero.
#endif
#define SPINDLE_PWM_OFF_VALUE 0
#define SPINDLE_PWM_RANGE (SPINDLE_PWM_MAX_VALUE-SPINDLE_PWM_MIN_VALUE)

//Control Digital Pin 8
#define SPINDLE_TCCRA_REGISTER TCCR4A
#define SPINDLE_TCCRB_REGISTER TCCR4B
#define SPINDLE_OCR_REGISTER OCR4C
#define SPINDLE_COMB_BIT COM4C1

// 1/8 Prescaler, 16-bit Fast PWM mode
#define SPINDLE_TCCRA_INIT_MASK ((1<<WGM40) | (1<<WGM41))
#define SPINDLE_TCCRB_INIT_MASK ((1<<WGM42) | (1<<WGM43) | (1<<CS41))
```

```

#define SPINDLE_OCRA_REGISTER OCR4A // 16-bit Fast PWM mode requires top reset value stored here.
#define SPINDLE_OCRA_TOP_VALUE 0x400 // PWM counter reset value. Should be the same as PWM_MAX_VALUE

// Define spindle output pins.
#define SPINDLE_PWM_DDR DDRH
#define SPINDLE_PWM_PORT PORTH
#define SPINDLE_PWM_BIT 5 // MEGA2560 Digital Pin 8

#elif defined (SPINDLE_PWM_ON_D6)

// Set Timer up to use TIMER4C which is attached to Digital Pin 6 - Ramps Servo 2
#define SPINDLE_PWM_MAX_VALUE 255.0 // Translates to about 1.9 kHz PWM frequency at 1/8 prescaler
#ifndef SPINDLE_PWM_MIN_VALUE
#define SPINDLE_PWM_MIN_VALUE 1 // Must be greater than zero.
#endif
#define SPINDLE_PWM_OFF_VALUE 0
#define SPINDLE_PWM_RANGE (SPINDLE_PWM_MAX_VALUE-SPINDLE_PWM_MIN_VALUE)

//Control Digital Pin 6 which is Servo 2 signal pin on Ramps 1.4 board
#define SPINDLE_TCCRA_REGISTER TCCR4A
#define SPINDLE_TCCRB_REGISTER TCCR4B
#define SPINDLE_OCR_REGISTER OCR4A
#define SPINDLE_COMB_BIT COM4A1

// 1/8 Prescaler, 8-bit Fast PWM mode
#define SPINDLE_TCCRA_INIT_MASK (1<<WGM41)
#define SPINDLE_TCCRB_INIT_MASK ((1<<WGM42) | (1<<WGM43) | (1<<CS41))
#define SPINDLE_OCRA_REGISTER ICR4 // 8-bit Fast PWM mode requires top reset value stored here.
#define SPINDLE_OCRA_TOP_VALUE 0xFF // PWM counter reset value. Should be the same as PWM_MAX_VALUE

// Define spindle output pins.
#define SPINDLE_PWM_DDR DDRH
#define SPINDLE_PWM_PORT PORTH
#define SPINDLE_PWM_BIT 3 // MEGA2560 Digital Pin 6

#elif defined (SPINDLE_PWM_ON_D9)

// Set Timer up to use TIMER2B which is attached to Digital Pin 9 - Ramps D9
#define SPINDLE_PWM_MAX_VALUE 255.0 // Translates to about 1.9 kHz PWM frequency at 1/8 prescaler
#ifndef SPINDLE_PWM_MIN_VALUE
#define SPINDLE_PWM_MIN_VALUE 1 // Must be greater than zero.
#endif
#define SPINDLE_PWM_OFF_VALUE 0
#define SPINDLE_PWM_RANGE (SPINDLE_PWM_MAX_VALUE-SPINDLE_PWM_MIN_VALUE)

//Control Digital Pin 9
#define SPINDLE_TCCRA_REGISTER TCCR2A
#define SPINDLE_TCCRB_REGISTER TCCR2B
#define SPINDLE_OCR_REGISTER OCR2B
#define SPINDLE_COMB_BIT COM2B1

```

```

// 1/8 Prescaler, 8-bit Fast PWM mode
#define SPINDLE_TCCRA_INIT_MASK ((1<<WGM20) | (1<<WGM21))
#define SPINDLE_TCCRB_INIT_MASK ((1<<WGM22) | (1<<CS22))
#define SPINDLE_OCRA_REGISTER OCR2A // 8-bit Fast PWM mode requires top reset value stored here.
#define SPINDLE_OCRA_TOP_VALUE 0xFF // PWM counter reset value. Should be the same as PWM_MAX_VALUE

// Define spindle output pins.
#define SPINDLE_PWM_DDR DDRH
#define SPINDLE_PWM_PORT PORTH
#define SPINDLE_PWM_BIT 6 // MEGA2560 Digital Pin 9

#else
#error "You must define SPINDLE_PWM_ON_D8 or SPINDLE_PWM_ON_D6 or SPINDLE_PWM_ON_D9 in config.h!"
#endif

#ifndef SEPARATE_SPINDLE_LASER_PIN

#if defined (LASER_PWM_ON_D6)

// Set Timer up to use TIMER4C which is attached to Digital Pin 6 - Ramps Servo 2
#define LASER_PWM_MAX_VALUE 255.0 // Translates to about 1.9 kHz PWM frequency at 1/8 prescaler
#ifndef LASER_PWM_MIN_VALUE
#define LASER_PWM_MIN_VALUE 1 // Must be greater than zero.
#endif
#define LASER_PWM_OFF_VALUE 0
#define LASER_PWM_RANGE (LASER_PWM_MAX_VALUE-LASER_PWM_MIN_VALUE)

//Control Digital Pin 6 which is Servo 2 signal pin on Ramps 1.4 board
#define LASER_TCCRA_REGISTER TCCR4A
#define LASER_TCCRB_REGISTER TCCR4B
#define LASER_OCR_REGISTER OCR4A
#define LASER_COMB_BIT COM4A1

// 1/8 Prescaler, 8-bit Fast PWM mode
#define LASER_TCCRA_INIT_MASK (1<<WGM41)
#define LASER_TCCRB_INIT_MASK ((1<<WGM42) | (1<<WGM43) | (1<<CS41))
#define LASER_OCRA_REGISTER IC4 // 8-bit Fast PWM mode requires top reset value stored here.
#define LASER_OCRA_TOP_VALUE 0xFF // PWM counter reset value. Should be the same as PWM_MAX_VALUE in

// Define spindle LASER pins.
#define LASER_PWM_DDR DDRH
#define LASER_PWM_PORT PORTH
#define LASER_PWM_BIT 3 // MEGA2560 Digital Pin 6

#elif defined (LASER_PWM_ON_D8)

// Set Timer up to use TIMER4B which is attached to Digital Pin 8 - Ramps 1.4 12v LASER with heat sink
#define LASER_PWM_MAX_VALUE 1024.0 // Translates to about 1.9 kHz PWM frequency at 1/8 prescaler
#ifndef LASER_PWM_MIN_VALUE
#define LASER_PWM_MIN_VALUE 1 // Must be greater than zero.

```

```

#endif
#define LASER_PWM_OFF_VALUE 0
#define LASER_PWM_RANGE (LASER_PWM_MAX_VALUE-LASER_PWM_MIN_VALUE)

//Control Digital Pin 8
#define LASER_TCCRA_REGISTER TCCR4A
#define LASER_TCCRB_REGISTER TCCR4B
#define LASER_OCR_REGISTER OCR4C
#define LASER_COMB_BIT COM4C1

// 1/8 Prescaler, 16-bit Fast PWM mode
#define LASER_TCCRA_INIT_MASK ((1<<WGM40) | (1<<WGM41))
#define LASER_TCCRB_INIT_MASK ((1<<WGM42) | (1<<WGM43) | (1<<CS41))
#define LASER_OCRA_REGISTER OCR4A // 16-bit Fast PWM mode requires top reset value stored here.
#define LASER_OCRA_TOP_VALUE 0x400 // PWM counter reset value. Should be the same as PWM_MAX_VALUE i

// Define spindle LASER pins.
#define LASER_PWM_DDR DDRH
#define LASER_PWM_PORT PORTH
#define LASER_PWM_BIT 5 // MEGA2560 Digital Pin 8

#elif defined (LASER_PWM_ON_D9)

// Set Timer up to use TIMER2B which is attached to Digital Pin 9 - Ramps D9
#define LASER_PWM_MAX_VALUE 255.0 // Translates to about 1.9 kHz PWM frequency at 1/8 prescaler
#ifndef LASER_PWM_MIN_VALUE
#define LASER_PWM_MIN_VALUE 1 // Must be greater than zero.
#endif
#define LASER_PWM_OFF_VALUE 0
#define LASER_PWM_RANGE (LASER_PWM_MAX_VALUE-LASER_PWM_MIN_VALUE)

//Control Digital Pin 9
#define LASER_TCCRA_REGISTER TCCR2A
#define LASER_TCCRB_REGISTER TCCR2B
#define LASER_OCR_REGISTER OCR2B
#define LASER_COMB_BIT COM2B1

// 1/8 Prescaler, 8-bit Fast PWM mode
#define LASER_TCCRA_INIT_MASK ((1<<WGM20) | (1<<WGM21))
#define LASER_TCCRB_INIT_MASK ((1<<WGM22) | (1<<CS22))
#define LASER_OCRA_REGISTER OCR2A // 8-bit Fast PWM mode requires top reset value stored here.
#define LASER_OCRA_TOP_VALUE 0xFF // PWM counter reset value. Should be the same as PWM_MAX_VALUE i

// Define spindle LASER pins.
#define LASER_PWM_DDR DDRH
#define LASER_PWM_PORT PORTH
#define LASER_PWM_BIT 6 // MEGA2560 Digital Pin 9

#else
#error "LASER_PWM_ON_D8 or LASER_PWM_ON_D6 or LASER_PWM_ON_D9 must be defined in config.h with the
#endif

```

```
#endif // SEPARATE_SPINDLE_LASER_PIN
```

```
#ifdef USE_OUTPUT_PWM
```

```
#if defined (OUTPUT_PWM_ON_D9)
```

```
// Error if both spindle and analog output are defined on the same pin
```

```
#ifndef SPINDLE_PWM_ON_D9
```

```
#error "Spindle is already defined on D9, you cant use the same D9 pin for analog output!"
```

```
#endif
```

```
// Set Timer up to use TIMER2B which is attached to Digital Pin 9 - Ramps D9
```

```
#define OUTPUT_PWM_MAX_VALUE 255.0 // Translates to about 1.9 kHz PWM frequency at 1/8 prescaler
```

```
#ifndef OUTPUT_PWM_MIN_VALUE
```

```
#define OUTPUT_PWM_MIN_VALUE 1 // Must be greater than zero.
```

```
#endif
```

```
#define OUTPUT_PWM_OFF_VALUE 0
```

```
#define OUTPUT_PWM_RANGE (OUTPUT_PWM_MAX_VALUE-OUTPUT_PWM_MIN_VALUE)
```

```
//Control Digital Pin 9
```

```
#define OUTPUT_TCCRA_REGISTER TCCR2A
```

```
#define OUTPUT_TCCRB_REGISTER TCCR2B
```

```
#define OUTPUT_OCR_REGISTER OCR2B
```

```
#define OUTPUT_COMB_BIT COM2B1
```

```
// 1/8 Prescaler, 8-bit Fast PWM mode
```

```
#define OUTPUT_TCCRA_INIT_MASK ((1<<WGM20) | (1<<WGM21))
```

```
#define OUTPUT_TCCRB_INIT_MASK ((1<<WGM22) | (1<<CS22))
```

```
#define OUTPUT_OCRA_REGISTER OCR2A // 8-bit Fast PWM mode requires top reset value stored here.
```

```
#define OUTPUT_OCRA_TOP_VALUE 0xFF // PWM counter reset value. Should be the same as PWM_MAX_VALUE
```

```
// Define spindle output pins.
```

```
#define OUTPUT_PWM_DDR DDRH
```

```
#define OUTPUT_PWM_PORT PORTH
```

```
#define OUTPUT_PWM_BIT 6 // MEGA2560 Digital Pin 9
```

```
#elif defined (OUTPUT_PWM_ON_D8)
```

```
// Error if both spindle and analog output are defined on the same pin
```

```
#ifndef SPINDLE_PWM_ON_D8
```

```
#error "Spindle is already defined on D8, you cant use the same D8 pin for analog output!"
```

```
#endif
```

```
// Error when analog output and spindle use the same timer
```

```
#ifndef SPINDLE_PWM_ON_D6
```

```
#error "Spindle is defined on D6 which use the same timer than D8, you cant use D8 pin for analog output!"
```

```
#endif
```

```
// Set Timer up to use TIMER4B which is attached to Digital Pin 8 - Ramps 1.4 12v output with heat sink
```

```
#define OUTPUT_PWM_MAX_VALUE 1024.0 // Translates to about 1.9 kHz PWM frequency at 1/8 prescaler
```

```
#ifndef OUTPUT_PWM_MIN_VALUE
```

```
#define OUTPUT_PWM_MIN_VALUE 1 // Must be greater than zero.
```

```
#endif
```

```

#define OUTPUT_PWM_OFF_VALUE 0
#define OUTPUT_PWM_RANGE (OUTPUT_PWM_MAX_VALUE-OUTPUT_PWM_MIN_VALUE)

//Control Digital Pin 8
#define OUTPUT_TCCRA_REGISTER TCCR4A
#define OUTPUT_TCCRB_REGISTER TCCR4B
#define OUTPUT_OCR_REGISTER OCR4C
#define OUTPUT_COMB_BIT COM4C1

// 1/8 Prescaler, 16-bit Fast PWM mode
#define OUTPUT_TCCRA_INIT_MASK ((1<<WGM40) | (1<<WGM41))
#define OUTPUT_TCCRB_INIT_MASK ((1<<WGM42) | (1<<WGM43) | (1<<CS41))
#define OUTPUT_OCRA_REGISTER OCR4A // 16-bit Fast PWM mode requires top reset value stored here.
#define OUTPUT_OCRA_TOP_VALUE 0x400 // PWM counter reset value. Should be the same as PWM_MAX_VALUE

// Define spindle output pins.
#define OUTPUT_PWM_DDR DDRH
#define OUTPUT_PWM_PORT PORTH
#define OUTPUT_PWM_BIT 5 // MEGA2560 Digital Pin 8

#elif defined (OUTPUT_PWM_ON_D6)

// Error if both spindle and analog output are defined on the same pin
#ifndef SPINDLE_PWM_ON_D6
#error "Spindle is already defined on D6, you cant use the same D6 pin for analog output!"
#endif
// Error when analog output and spindle use the same timer
#ifndef SPINDLE_PWM_ON_D8
#error "Spindle is defined on D8 which use the same timer than D6, you cant use D6 pin for analog output!"
#endif
// Set Timer up to use TIMER4C which is attached to Digital Pin 6 - Ramps Servo 2
#define OUTPUT_PWM_MAX_VALUE 255.0 // Translates to about 1.9 kHz PWM frequency at 1/8 prescaler
#ifndef OUTPUT_PWM_MIN_VALUE
#define OUTPUT_PWM_MIN_VALUE 1 // Must be greater than zero.
#endif
#define OUTPUT_PWM_OFF_VALUE 0
#define OUTPUT_PWM_RANGE (OUTPUT_PWM_MAX_VALUE-OUTPUT_PWM_MIN_VALUE)

//Control Digital Pin 6 which is Servo 2 signal pin on Ramps 1.4 board
#define OUTPUT_TCCRA_REGISTER TCCR4A
#define OUTPUT_TCCRB_REGISTER TCCR4B
#define OUTPUT_OCR_REGISTER OCR4A
#define OUTPUT_COMB_BIT COM4A1

// 1/8 Prescaler, 8-bit Fast PWM mode
#define OUTPUT_TCCRA_INIT_MASK (1<<WGM41)
#define OUTPUT_TCCRB_INIT_MASK ((1<<WGM42) | (1<<WGM43) | (1<<CS41))
#define OUTPUT_OCRA_REGISTER ICR4 // 8-bit Fast PWM mode requires top reset value stored here.
#define OUTPUT_OCRA_TOP_VALUE 0xFF // PWM counter reset value. Should be the same as PWM_MAX_VALUE

// Define spindle output pins.

```

```

#define OUTPUT_PWM_DDR DDRH
#define OUTPUT_PWM_PORT PORTH
#define OUTPUT_PWM_BIT 3 // MEGA2560 Digital Pin 6

#else
#error "OUTPUT_PWM_ON_D9 or OUTPUT_PWM_ON_D8 or OUTPUT_PWM_ON_D6 must be defined in config.h with"
#endif

#endif // USE_OUTPUT_PWM

#endif // CPU_MAP_2560_RAMPS_BOARD

#endif

```

File: defaults.h

Directory: C:/Users/sense/Downloads/grbl-Mega-5X-edge/grbl-Mega-5X-edge/grbl

```

/*
defaults.h - defaults settings configuration file
Part of Grbl

```

Copyright (c) 2017-2022 Gauthier Briere
 Copyright (c) 2012-2016 Sungeun K. Jeon for Gnea Research LLC

Grbl is free software: you can redistribute it and/or modify
 it under the terms of the GNU General Public License as published by
 the Free Software Foundation, either version 3 of the License, or
 (at your option) any later version.

Grbl is distributed in the hope that it will be useful,
 but WITHOUT ANY WARRANTY; without even the implied warranty of
 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 GNU General Public License for more details.

You should have received a copy of the GNU General Public License
 along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

```

*/

```

```

/* The defaults.h file serves as a central default settings selector for different machine
types, from DIY CNC mills to CNC conversions of off-the-shelf machines. The settings
files listed here are supplied by users, so your results may vary. However, this should
give you a good starting point as you get to know your machine and tweak the settings for
your nefarious needs.

```

```

NOTE: Ensure one and only one of these DEFAULTS_XXX values is defined in config.h */

```

```

#ifndef defaults_h

```

```

#ifndef DEFAULTS_GENERIC

```

```

// Generic conservative settings for a RAMP CNC machine. You must update these yourself.
// Keep in mind that Grbl is highly efficient and settings can be significantly different.
// Especially when optimizing for a different CNC task like going from 3d printing to
// CNC milling or laser cutting. Unlike Marlin, these defaults are only applied when the
// EEPROM is explicitly wiped, either by a `$RST=*` command or Grbl detecting a settings
// version type change (not frequent).
#define MICROSTEPS_AXIS1 4    // Microstepping = 1/4 pas
#define STEP_REVS_AXIS1 200   // Moteurs à 200 pas par tour
#define UNIT_PER_REV_AXIS1 2.0 // 2mm
#define DEFAULT_AXIS1_STEPS_PER_UNIT (MICROSTEPS_AXIS1*STEP_REVS_AXIS1/UNIT_PER_REV_AXIS1) //
#define MICROSTEPS_AXIS2 4    // Microstepping = 1/4 pas
#define STEP_REVS_AXIS2 200   // Moteurs à 200 pas par tour
#define UNIT_PER_REV_AXIS2 2.0 // 2mm
#define DEFAULT_AXIS2_STEPS_PER_UNIT (MICROSTEPS_AXIS2*STEP_REVS_AXIS2/UNIT_PER_REV_AXIS2)
#define MICROSTEPS_AXIS3 4    // Microstepping = 1/4 pas
#define STEP_REVS_AXIS3 200   // Moteurs à 200 pas par tour
#define UNIT_PER_REV_AXIS3 2.0 // 2mm
#define DEFAULT_AXIS3_STEPS_PER_UNIT (MICROSTEPS_AXIS3*STEP_REVS_AXIS3/UNIT_PER_REV_AXIS3)
#define STEP_MAX_FREQUENCY 5000
#define SECONDS_PER_MINUTE 60
// #define DEFAULT_AXIS1_MAX_RATE 9000.0 // 9000 mm/min = 9000/60 = 150 mm/sec
#define DEFAULT_AXIS1_MAX_RATE (STEP_MAX_FREQUENCY/DEFAULT_AXIS1_STEPS_PER_UNIT*SECONDS_
// #define DEFAULT_AXIS2_MAX_RATE 9000.0 // 9000 mm/min = 9000/60 = 150 mm/sec
#define DEFAULT_AXIS2_MAX_RATE (STEP_MAX_FREQUENCY/DEFAULT_AXIS2_STEPS_PER_UNIT*SECONDS_
// #define DEFAULT_AXIS3_MAX_RATE 300.0 // 300 mm/min = 300/60 = 5 mm/sec
#define DEFAULT_AXIS3_MAX_RATE (STEP_MAX_FREQUENCY/DEFAULT_AXIS3_STEPS_PER_UNIT*SECONDS_
#define DEFAULT_AXIS1_ACCELERATION (50.0*60*60) // 300*60*60 mm/min^2 = 300 mm/sec^2
#define DEFAULT_AXIS2_ACCELERATION (50.0*60*60) // 300*60*60 mm/min^2 = 300 mm/sec^2
#define DEFAULT_AXIS3_ACCELERATION (50.0*60*60) // 100*60*60 mm/min^2 = 100 mm/sec^2
#define DEFAULT_AXIS1_MAX_TRAVEL 400.0 // mm
#define DEFAULT_AXIS2_MAX_TRAVEL 200.0 // mm
#define DEFAULT_AXIS3_MAX_TRAVEL 200.0 // mm
#if N_AXIS > 3
  #if AXIS_4_NAME != AXIS_1_NAME && AXIS_4_NAME != AXIS_2_NAME && AXIS_4_NAME != AXIS_3_NAME
    #define DEFAULT_AXIS4_STEPS_PER_UNIT 8.888889 // Direct drive : (200 pas par tours * 1/16 microsteps)/360°
    #define DEFAULT_AXIS4_MAX_RATE 1440 // °/mn
    #define DEFAULT_AXIS4_ACCELERATION (50.0*60*60) // 100*60*60 mm/min^2 = 100 mm/sec^2
    #define DEFAULT_AXIS4_MAX_TRAVEL 360.0 // °
  #elif AXIS_4_NAME == AXIS_1_NAME
    #define DEFAULT_AXIS4_STEPS_PER_UNIT DEFAULT_AXIS1_STEPS_PER_UNIT
    #define DEFAULT_AXIS4_MAX_RATE DEFAULT_AXIS1_MAX_RATE
    #define DEFAULT_AXIS4_ACCELERATION DEFAULT_AXIS1_ACCELERATION
    #define DEFAULT_AXIS4_MAX_TRAVEL DEFAULT_AXIS1_MAX_TRAVEL
  #elif AXIS_4_NAME == AXIS_2_NAME
    #define DEFAULT_AXIS4_STEPS_PER_UNIT DEFAULT_AXIS2_STEPS_PER_UNIT
    #define DEFAULT_AXIS4_MAX_RATE DEFAULT_AXIS2_MAX_RATE
    #define DEFAULT_AXIS4_ACCELERATION DEFAULT_AXIS2_ACCELERATION
    #define DEFAULT_AXIS4_MAX_TRAVEL DEFAULT_AXIS2_MAX_TRAVEL
  #else
    #define DEFAULT_AXIS4_STEPS_PER_UNIT DEFAULT_AXIS3_STEPS_PER_UNIT
    #define DEFAULT_AXIS4_MAX_RATE DEFAULT_AXIS3_MAX_RATE

```

```

#define DEFAULT_AXIS4_ACCELERATION DEFAULT_AXIS3_ACCELERATION
#define DEFAULT_AXIS4_MAX_TRAVEL DEFAULT_AXIS3_MAX_TRAVEL
#endif
#endif
#if N_AXIS > 4
    #if AXIS_5_NAME != AXIS_1_NAME && AXIS_5_NAME != AXIS_2_NAME && AXIS_5_NAME != AXIS_3_NAME && \
        AXIS_5_NAME != AXIS_4_NAME
        #define DEFAULT_AXIS5_STEPS_PER_UNIT 8.888889 // Direct drive : (200 pas par tours * 1/16 microsteps)/360°
        #define DEFAULT_AXIS5_MAX_RATE 1440 // °/mn
        #define DEFAULT_AXIS5_ACCELERATION (50.0*60*60) // 100*60*60 mm/min^2 = 100 mm/sec^2
        #define DEFAULT_AXIS5_MAX_TRAVEL 180.0 // °
    #elif AXIS_5_NAME == AXIS_1_NAME
        #define DEFAULT_AXIS5_STEPS_PER_UNIT DEFAULT_AXIS1_STEPS_PER_UNIT
        #define DEFAULT_AXIS5_MAX_RATE DEFAULT_AXIS1_MAX_RATE
        #define DEFAULT_AXIS5_ACCELERATION DEFAULT_AXIS1_ACCELERATION
        #define DEFAULT_AXIS5_MAX_TRAVEL DEFAULT_AXIS1_MAX_TRAVEL
    #elif AXIS_5_NAME == AXIS_2_NAME
        #define DEFAULT_AXIS5_STEPS_PER_UNIT DEFAULT_AXIS2_STEPS_PER_UNIT
        #define DEFAULT_AXIS5_MAX_RATE DEFAULT_AXIS2_MAX_RATE
        #define DEFAULT_AXIS5_ACCELERATION DEFAULT_AXIS2_ACCELERATION
        #define DEFAULT_AXIS5_MAX_TRAVEL DEFAULT_AXIS2_MAX_TRAVEL
    #elif AXIS_5_NAME == AXIS_3_NAME
        #define DEFAULT_AXIS5_STEPS_PER_UNIT DEFAULT_AXIS3_STEPS_PER_UNIT
        #define DEFAULT_AXIS5_MAX_RATE DEFAULT_AXIS3_MAX_RATE
        #define DEFAULT_AXIS5_ACCELERATION DEFAULT_AXIS3_ACCELERATION
        #define DEFAULT_AXIS5_MAX_TRAVEL DEFAULT_AXIS3_MAX_TRAVEL
    #else
        #define DEFAULT_AXIS5_STEPS_PER_UNIT DEFAULT_AXIS4_STEPS_PER_UNIT
        #define DEFAULT_AXIS5_MAX_RATE DEFAULT_AXIS4_MAX_RATE
        #define DEFAULT_AXIS5_ACCELERATION DEFAULT_AXIS4_ACCELERATION
        #define DEFAULT_AXIS5_MAX_TRAVEL DEFAULT_AXIS4_MAX_TRAVEL
    #endif
#endif
#if N_AXIS > 5
    #if AXIS_6_NAME != AXIS_1_NAME && AXIS_6_NAME != AXIS_2_NAME && \
        AXIS_6_NAME != AXIS_3_NAME && AXIS_6_NAME != AXIS_4_NAME && AXIS_6_NAME != AXIS_5_NAME
        #define DEFAULT_AXIS6_STEPS_PER_UNIT 8.888889 // Direct drive : (200 pas par tours * 1/16 microsteps)/360°
        #define DEFAULT_AXIS6_MAX_RATE 1440 // °/mn
        #define DEFAULT_AXIS6_ACCELERATION (50.0*60*60) // 100*60*60 mm/min^2 = 100 mm/sec^2
        #define DEFAULT_AXIS6_MAX_TRAVEL 180.0 // °
    #elif AXIS_6_NAME == AXIS_1_NAME
        #define DEFAULT_AXIS6_STEPS_PER_UNIT DEFAULT_AXIS1_STEPS_PER_UNIT
        #define DEFAULT_AXIS6_MAX_RATE DEFAULT_AXIS1_MAX_RATE
        #define DEFAULT_AXIS6_ACCELERATION DEFAULT_AXIS1_ACCELERATION
        #define DEFAULT_AXIS6_MAX_TRAVEL DEFAULT_AXIS1_MAX_TRAVEL
    #elif AXIS_6_NAME == AXIS_2_NAME
        #define DEFAULT_AXIS6_STEPS_PER_UNIT DEFAULT_AXIS2_STEPS_PER_UNIT
        #define DEFAULT_AXIS6_MAX_RATE DEFAULT_AXIS2_MAX_RATE
        #define DEFAULT_AXIS6_ACCELERATION DEFAULT_AXIS2_ACCELERATION
        #define DEFAULT_AXIS6_MAX_TRAVEL DEFAULT_AXIS2_MAX_TRAVEL
    #elif AXIS_6_NAME == AXIS_3_NAME

```

```

#define DEFAULT_AXIS6_STEPS_PER_UNIT DEFAULT_AXIS3_STEPS_PER_UNIT
#define DEFAULT_AXIS6_MAX_RATE DEFAULT_AXIS3_MAX_RATE
#define DEFAULT_AXIS6_ACCELERATION DEFAULT_AXIS3_ACCELERATION
#define DEFAULT_AXIS6_MAX_TRAVEL DEFAULT_AXIS3_MAX_TRAVEL
#elif AXIS_6_NAME == AXIS_4_NAME
#define DEFAULT_AXIS6_STEPS_PER_UNIT DEFAULT_AXIS4_STEPS_PER_UNIT
#define DEFAULT_AXIS6_MAX_RATE DEFAULT_AXIS4_MAX_RATE
#define DEFAULT_AXIS6_ACCELERATION DEFAULT_AXIS4_ACCELERATION
#define DEFAULT_AXIS6_MAX_TRAVEL DEFAULT_AXIS4_MAX_TRAVEL
#else
#define DEFAULT_AXIS6_STEPS_PER_UNIT DEFAULT_AXIS5_STEPS_PER_UNIT
#define DEFAULT_AXIS6_MAX_RATE DEFAULT_AXIS5_MAX_RATE
#define DEFAULT_AXIS6_ACCELERATION DEFAULT_AXIS5_ACCELERATION
#define DEFAULT_AXIS6_MAX_TRAVEL DEFAULT_AXIS5_MAX_TRAVEL
#endif
#endif

#define DEFAULT_SPINDLE_RPM_MAX 12000 // rpm
#define DEFAULT_SPINDLE_RPM_MIN 550.0 // rpm
#ifndef SEPARATE_SPINDLE_LASER_PIN
#define DEFAULT_LASER_MAX 100
#define DEFAULT_LASER_MIN 1
#endif
#ifndef USE_OUTPUT_PWM
#if defined(OUTPUT_PWM_ON_D9) || defined(OUTPUT_PWM_ON_D8)
#define DEFAULT_OUTPUT_MAX 12.0 // volts
#define DEFAULT_OUTPUT_MIN 1.4 // volts
#else // (OUTPUT_PWM_ON_D6)
#define DEFAULT_OUTPUT_MAX 5.0 // volts
#define DEFAULT_OUTPUT_MIN 0.04 // volts
#endif
#endif
#endif

#define DEFAULT_STEP_PULSE_MICROSECONDS 10
#define DEFAULT_STEPPING_INVERT_MASK 0
#define DEFAULT_DIRECTION_INVERT_MASK 0
#define DEFAULT_STEPPER_IDLE_LOCK_TIME 254 // msec (0-254, 255 keeps steppers enabled)
#define DEFAULT_STATUS_REPORT_MASK 1 // MPos enabled
#define DEFAULT_JUNCTION_DEVIATION 0.02 // mm
#define DEFAULT_ARC_TOLERANCE 0.002 // mm
#define DEFAULT_REPORT_INCHES 0 // false
#define DEFAULT_INVERT_ST_ENABLE 0 // false
#define DEFAULT_INVERT_LIMIT_PINS 0 // false
#define DEFAULT_SOFT_LIMIT_ENABLE 1 // true
#define DEFAULT_HARD_LIMIT_ENABLE 0 // false
#define DEFAULT_INVERT_PROBE_PIN 0 // false
#define DEFAULT_LASER_MODE 0 // false
#define DEFAULT_HOMING_ENABLE 1 // true
#define DEFAULT_HOMING_DIR_MASK 0 // move positive dir
#define DEFAULT_HOMING_FEED_RATE 25.0 // mm/min
#define DEFAULT_HOMING_SEEK_RATE 250.0 // mm/min
#define DEFAULT_HOMING_DEBOUNCE_DELAY 250 // msec (0-65k)
#define DEFAULT_HOMING_PULLOFF 5.0 // mm

```

```
#endif
```

```
#endif
```

File: grbl.h

Directory: C:/Users/sense/Downloads/grbl-Mega-5X-edge/grbl-Mega-5X-edge/grbl

```
/*
```

```
grbl.h - main Grbl include file
```

```
Part of Grbl
```

Copyright (c) 2017-2022 Gauthier Briere

Copyright (c) 2015-2016 Sungeun K. Jeon for Gnea Research LLC

Grbl is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

Grbl is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

```
*/
```

```
#ifndef grbl_h
```

```
#define grbl_h
```

```
// Grbl versioning system
```

```
#define GRBL_VERSION "1.2h"
```

```
#define GRBL_VERSION_BUILD "20220109"
```

```
// Define standard libraries used by Grbl.
```

```
#include <avr/io.h>
```

```
#include <avr/pgmspace.h>
```

```
#include <avr/interrupt.h>
```

```
#include <avr/wdt.h>
```

```
#include <util/delay.h>
```

```
#include <math.h>
```

```
#include <inttypes.h>
```

```
#include <string.h>
```

```
#include <stdlib.h>
```

```
#include <stdint.h>
```

```
#include <stdbool.h>
```

```
// Define the Grbl system include files. NOTE: Do not alter organization.
```

```

#include "config.h"
#include "nuts_bolts.h"
#include "settings.h"
#include "system.h"
#include "defaults.h"
#include "cpu_map.h"
#include "planner.h"
#include "coolant_control.h"
#include "digital_control.h"
#include "analog_control.h"
#include "eeprom.h"
#include "gcode.h"
#include "limits.h"
#include "motion_control.h"
#include "planner.h"
#include "print.h"
#include "probe.h"
#include "protocol.h"
#include "report.h"
#include "serial.h"
#include "spindle_control.h"
#include "stepper.h"
#include "jog.h"
#include "sleep.h"

// -----
// COMPILE-TIME ERROR CHECKING OF DEFINE VALUES:

#ifndef HOMING_CYCLE_0
    #error "Required HOMING_CYCLE_0 not defined."
#endif

#if defined(PARKING_ENABLE)
    #if defined(HOMING_FORCE_SET_ORIGIN)
        #error "HOMING_FORCE_SET_ORIGIN is not supported with PARKING_ENABLE at this time."
    #endif
#endif

#if defined(ENABLE_PARKING_OVERRIDE_CONTROL)
    #if !defined(PARKING_ENABLE)
        #error "ENABLE_PARKING_OVERRIDE_CONTROL must be enabled with PARKING_ENABLE."
    #endif
#endif

#if defined(SPINDLE_PWM_MIN_VALUE)
    #if !(SPINDLE_PWM_MIN_VALUE > 0)
        #error "SPINDLE_PWM_MIN_VALUE must be greater than zero."
    #endif
#endif

#if (REPORT_WCO_REFRESH_BUSY_COUNT < REPORT_WCO_REFRESH_IDLE_COUNT)

```

```

#error "WCO busy refresh is less than idle refresh."
#endif
#if (REPORT_OVR_REFRESH_BUSY_COUNT < REPORT_OVR_REFRESH_IDLE_COUNT)
#error "Override busy refresh is less than idle refresh."
#endif
#if (REPORT_WCO_REFRESH_IDLE_COUNT < 2)
#error "WCO refresh must be greater than one."
#endif
#if (REPORT_OVR_REFRESH_IDLE_COUNT < 1)
#error "Override refresh must be greater than zero."
#endif

// -----

#endif

```

File: stepper.h

Directory: C:/Users/sense/Downloads/grbl-Mega-5X-edge/grbl-Mega-5X-edge/grbl

```

/*
stepper.h - stepper motor driver: executes motion plans of planner.c using the stepper motors
Part of Grbl

```

Copyright (c) 2017-2022 Gauthier Briere
 Copyright (c) 2011-2016 Sungeun K. Jeon for Gnea Research LLC
 Copyright (c) 2009-2011 Simen Svale Skogsrud

Grbl is free software: you can redistribute it and/or modify
 it under the terms of the GNU General Public License as published by
 the Free Software Foundation, either version 3 of the License, or
 (at your option) any later version.

Grbl is distributed in the hope that it will be useful,
 but WITHOUT ANY WARRANTY; without even the implied warranty of
 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 GNU General Public License for more details.

You should have received a copy of the GNU General Public License
 along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

```

*/

```

```

#ifndef stepper_h
#define stepper_h

```

```

#ifndef SEGMENT_BUFFER_SIZE
#define SEGMENT_BUFFER_SIZE 10
#endif

```

```

// Initialize and setup the stepper motor subsystem

```

```
void stepper_init();

// Enable steppers, but cycle does not start unless called by motion control or realtime command.
void st_wake_up();

// Immediately disables steppers
void st_go_idle();

// Generate the step and direction port invert masks.
void st_generate_step_dir_invert_masks();

// Reset the stepper subsystem variables
void st_reset();

// Changes the run state of the step segment buffer to execute the special parking motion.
void st_parking_setup_buffer();

// Restores the step segment buffer to the normal run state after a parking motion.
void st_parking_restore_buffer();

// Reloads step segment buffer. Called continuously by realtime execution system.
void st_prep_buffer();

// Called by planner_recalculate() when the executing block is updated by the new plan.
void st_update_plan_block_parameters();

// Called by realtime status reporting if realtime rate reporting is enabled in config.h.
float st_get_realtime_rate();

#endif
```